KU LEUVEN kulak

Faculty of Science & Technology Kulak

Introduction to 2D physics engines

Constraint-based rigid-body simulations

Casper Vermeeren

Physics

2026

Introduction to 2D physics engines

Casper Vermeeren

2026

Contents

Introduction

If you're reading this, there's a large chance you've played a game before. If you haven't, you fall into the small non-overlap between science enthusiasts and gamers. We take it for granted how easy it is to open up a game or some software and see squares, rectangles, circles, bouncing around and colliding with each other realistically. Even most game developers blindly accept the existence of these simulations: they tick the box that says "collision" and move on with creating their game. But what actually goes into the underlying physics engine?

Because the visual output is so intuitive, you might assume creating it yourself would be simple. But while it certainly isn't the world's most daunting task, creating a 2D physics engine from scratch is no light work. To do it correctly requires an understanding of classical mechanics, calculus, linear algebra, and software design. This course was created with the intention of combining all this knowledge and derivation into one journey. The journey from a completely empty project with no external libraries to a full rigid-body physics engine. Let's dive in.

Prerequisites

This course was aimed at university students in exact sciences at an undergraduate level or higher. While we will explain a lot of basic concepts in detail throughout the course, a good understanding of the following topics is recommended:

- Calculus: limits, derivatives, integrals, differential equations
- Classical mechanics: Newton's laws, forces, Lagrangian mechanics
- Linear algebra: vectors, matrices, linear systems, projection
- Computer science: basic programming, object-oriented programming

Chapter 1

Unconstrained dynamics

1.1 The basics

1.1.1 The state vector

The state vector is a tool used to describe one time-frame of the entire system. Given this vector, one should be able to perfectly reproduce an image of the current situation. From this description it thus also follows that the state vector and all of its components are - or can be - time-dependent. A two-dimensional system describing two particles would have a state vector with four components: 2 dimensions * 2 particles. A general 2D system with N particles has a state vector of length 2N.

$$\vec{X}(t) = \begin{pmatrix} x_1(t) \\ y_1(t) \\ x_2(t) \\ y_2(t) \\ \vdots \end{pmatrix}$$

All a physics engine should do is correctly identify the relations between $\vec{X}$, its derivative $\dot{\vec{X}}$, and the forces of the world. Throughout this course, it should become clear that each layer of motion (position, velocity, acceleration) handles changes differently and that, even if acceleration is usually all you need, separating these layers can be crucial in the numerical aspect of the simulation.

1.1.2 Forces and their components

Velocities change through acceleration, caused by forces. If we can calculate a force using some equation, it can easily be converted to an acceleration via $F = ma$. This acceleration, however, is still a vector. One must therefore be able to dynamically split the acceleration vector into the components a_x and a_y, such that they can be added as parts of the velocity derivative. This also requires the perpendicular x- and y-axes to be defined in advance.

We will generally cover two types of forces: external forces and internal forces. While not a one-to-one relation, you could also differentiate these as known forces and constraint forces. The former are forces we can calculate using a known equation, such as gravity ($F = mg$), spring

force ($F = kx$) or air resistance ($F = -bv$). The latter are harder to calculate, since they will only be known once the system has been entirely solved. Examples are normal force, tension force, friction force, and any other type of force that keeps objects fixed or in some shape. These constraint-dependent forces will be derived in more complex ways and will also be the main reason why our simulations are not exact - a system whose energy should be conserved in theory might not conserve it perfectly in practice because of these approximations.

1.2 Why not Lagrangian mechanics?

With these constraint-dependent caveats in mind, you could suggest using Lagrangian mechanics for our simulations. This is a great recommendation on the surface, as using the Lagrangian allows us to convert (holonomic) constraints into reductions in coordinates and a great way to derive the differential equations of motion. Furthermore, whatever Lagrangian mechanics outputs, will - under a proper integrator - deliver physically exact solutions.

The only problem is that this method is extremely situation-dependent. Major steps such as the choice of generalized coordinates and the expressions of kinetic and potential energy are hard to automate in code. If we had a magic box that we could throw a problem into and it spit out the necessary coordinates and accompanying energy expressions, then technically the remaining steps could be automated without issue. It just needs to compute the Lagrangian $T - V$ and solve a few partial derivatives to find the equations of motion, which can then be simulated with a numerical integrator.

Even with that, simulators using Lagrangian mechanics do of course exist. The downside of them is their lack of complete freedom in what you wish to create. Certain typical constraints like the double pendulum are often hard-coded, same for potentials like gravity. What we want is a fully dynamic system that can simulate anything you can dream of - the only limit being numerical accuracy and your computer's price tag.

1.3 Numerical time integrators

For the sake of completeness, we will briefly mention two popular integrators used to solve known differential equations. If you did manage to create that magic Lagrangian box we just mentioned, an RK4 integrator would be perfect for simulating the resulting differential equations.

1.3.1 Runge-Kutta

The basic idea behind a Runge-Kutta integrator is that we take a differential equation like $\frac{dy}{dx} = f(x, y)$ together with a starting point y_0 and a time step h ($x_{k+1} = x_k + h$), and state that

$$y_{k+1} = y_k + hf(x_k, y_k).$$

This means we're taking the slope of our solution at this point in time ($f(x_k, y_k)$) and using it to linearly approximate where the next point will be. But this is only first-order, meaning halving h will also only halve the error. For higher-order approximations, we have to first generalize this solution. Looking at the differential equation we're given and separating the variables, an exact solution would be

$$y_{k+1} = y_k + \int_{x_k}^{x_k+h} f(x, y)\, dx.$$

The crux of Runge-Kutta approximations is to numerically estimate that integral. We won't be covering numerical integration, so we'll assume the exact forms. Another way to look at it is starting from the current point, doing a few linear jumps to points between x_k and x_{k+1}, then combining all their slopes into some mega-average to use for the final full jump. Let's look at Runge-Kutta 4 (RK4), which is a fourth-order method meaning halving h causes the error to be divided by 16.

$$\begin{cases} b_1 = f(x_k, y_k) \\ b_2 = f(x_k + \frac{h}{2}, y_k + \frac{h}{2} b_1) \\ b_3 = f(x_k + \frac{h}{2}, y_k + \frac{h}{2} b_2) \\ b_4 = f(x_k + h, y_k + h b_3) \end{cases}$$

$$y_{k+1} = y_k + h \frac{b_1 + 2b_2 + 2b_3 + b_4}{6}$$

The trade-off between accuracy and computation time should now be very clear: RK4 gives tremendous results, conserving energy up to high decimals for physics simulations, but it also requires much more computation.[3]

1.3.2 Verlet

Another algorithm commonly used to solve Newtonian systems that conserve energy is Verlet integration - in our specific case: Velocity Verlet. The idea is that you have the system's potential energy $V(x)$, which can be used to determine the acceleration at any point. We then separate the position and velocity updates and make them hop over each other:

1. Calculate $v(t + \frac{1}{2}\Delta t) = v(t) + \frac{1}{2} a(t)\Delta t$
2. Calculate $x(t + \Delta t) = x(t) + v(t + \frac{1}{2}\Delta t)\Delta t$
3. Derive $a(t + \Delta t)$ from this new $x(t + \Delta t)$
4. Calculate $v(t + \Delta t) = v(t + \frac{1}{2}\Delta t) + \frac{1}{2} a(t + \Delta t)\Delta t$

The velocity jumps in half-steps, while the position jumps in full steps - the velocity is closer to the acceleration and thus carries more importance for the accuracy of the simulation. This oscillation and the use of multiple points in-between to inform a jump, should look quite similar to what we just saw in Runge-Kutta integration. The downside of a Verlet algorithm like this one is that it assumes acceleration is purely dependent on position and not velocity or other factors. Just like Runge-Kutta requiring an existing differential equation, it just isn't general enough for our purposes.

So what algorithm will we use? It won't be any of the ones covered here. In fact, it will be a very simple step in time, as simple as $x(t + \Delta t) = x(t) + v\Delta t$. The difference will be that the velocity v was gobbled up into the system's solving of forces and constraints, meaning no accelerations are necessary and the correct velocity is found directly. At that point such a direct position step shouldn't come as a surprise. Furthermore, we'll introduce a separate part of the solver that specifically creates a part of the velocity just to correct positional errors if they do occur.[19]

1.4 Known or external forces

Before covering the infamous internal (constraint) forces - which will require a full chapter - let's recap some basic external forces. As mentioned before, these forces have full algebraic equations that take in the current position and velocity, then immediately yield the acceleration caused.

1.4.1 Gravity

For any 2D simulation - unless we're dealing with microscopic particle physics - gravity is a no-brainer force to implement. While the general form discovered by Newton would be the most accurate, having every pair of bodies in your system act upon each other is asking for trouble - or as computer scientists would describe it: $O(n^2)$. That's why we only implement the most basic form:

$$F_y = -mg$$

This of course assumes that deep in the negative y direction there exists some Earth-like mass pulling everything down. For most rigid-body simulations this is more than enough, especially if we allow custom values of the gravitational acceleration g.

1.4.2 Air resistance

Similarly, simulating air resistance by spawning in millions of "air circles" that hit and slow down moving objects is incredibly inefficient. Luckily there exist approximations for resistance forces caused by moving through air. The most basic form we implemented is

$$\vec{F} = -b\vec{v}$$

where the force is directly opposing the current velocity, with a resistance factor b.

1.4.3 Springs

If two particles are connected by a spring, then the spring force on one of the two particles is given as

$$\vec{F} = k(l - l_0)\Delta\vec{r}$$

where $\Delta\vec{r}$ is the unit vector pointing from that particle (to whom the force applies) to the other particle. In the case of two particles, the length l can be calculated as

$$l = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

which corresponds to the length of the vector connecting the two particles. This unit vector can thus be found as

$$\Delta\vec{r} = \begin{pmatrix} \frac{x_2 - x_1}{l} & \frac{y_2 - y_1}{l} \end{pmatrix}$$

for the first particle and opposite for the second particle - which could also be derived from Newton's third law. In total, we thus find that

$$F_{1x} = k(l - l_0)\frac{x_2 - x_1}{l}$$

$$F_{1y} = k(l - l_0)\frac{y_2 - y_1}{l}$$

$$F_{2x} = k(l - l_0)\frac{x_1 - x_2}{l}$$

$$F_{2y} = k(l - l_0)\frac{y_1 - y_2}{l}.$$

Dividing by m_1 or m_2 to find components of acceleration gives us the full state vector for this setup, which can then be simulated.

Chapter 2

Principles of constraints

The second category of forces to model are the constraint or internal forces. As mentioned before, these are forces that arise from constraints or limits on how our system can move around. Examples include

- pendulums, where two points are connected by a rod of fixed length, where this fixing is the constraint.
- walls or boundaries, where particle positions are limited to a specific interval. These constraints are non-holonomic, meaning they are expressed as inequalities and not as equalities, which poses an even tougher challenge.
- fixed points, where an object (like a rod) is fixed at some point and can only rotate around this point.

The challenge in modeling these forces is that, unlike known or external forces, there are no textbook-defined equations for any of them. They depend heavily on the system itself and most notably on the motion it will perform. This means that to know the forces, we technically need to solve the motion. But to solve the motion, we also need the forces! In other words, the constraints themselves define the forces. This is not ground-breaking: even in basic Newtonian physics, things like normal force can be found by imposing that an object is at rest vertically and will thus need this upward force to equal downward gravity in magnitude.

To model forces that originate from constraints on our system, we'll need some very important mathematical toolkit. This will involve virtual displacements and virtual work, as well as how constraints should affect the balance of forces mathematically, and finally how to solve the implied optimization problems. In the third chapter we'll then generalize this for any amount of constraints using linear algebra.

2.1 Virtual displacements and virtual work

First, let's properly define what these 'virtual' shenanigans are.

Definition 2.1.1. A virtual displacement $\delta\vec{r}$ is an infinitesimal, time-independent, hypothetical displacement that respects all constraints of the system and has no relation to real displacements.[7]

At this point you should know what an infinitesimal is and how you can symbolically derive it. For example, in Lagrangian mechanics we say positions $\vec{r}_i$ depend on generalized coordinates $q_1, q_2, \ldots, q_n$ as well as time t, such that

$$d\vec{r}_i = \sum_{j=1}^{n} \frac{\partial \vec{r}_i}{\partial q_j} dq_j + \frac{\partial \vec{r}_i}{\partial t} dt.$$

The difference for a virtual displacement $\delta\vec{r}_i$ is that it happens in an instant; we do not take time dependency into account. Therefore we also neglect the actual time derivative, leaving us with

$$\delta\vec{r}_i = \sum_{j=1}^{n} \frac{\partial \vec{r}_i}{\partial q_j} \delta q_j.$$

Note that the change in the generalized coordinates has also been turned into a virtual change. Next let's introduce virtual work, as it's a simple extension.

Definition 2.1.2. Virtual work is the work done by some force over a virtual displacement.[7]

$$\delta W = \vec{F} \cdot \delta\vec{r}$$

In statics analysis - like for engineering applications - virtual work can be used to more efficiently find the size of internal forces. This is because in statics the total force is zero, such that the total virtual work for any allowed virtual displacement is also zero. Smart choices for the virtual displacement can then lead to some forces disappearing due to perpendicularity in the dot product, leaving relations between unknown forces and known forces.

We however are dealing with dynamic systems, they're not static at all. This sadly means the total force is no longer zero, but due to the way virtual displacements are defined, they will very often lead to ideal constraint forces.

Definition 2.1.3. For some dynamic system, a constraint force is ideal if the virtual work performed by it over an allowed virtual displacement is zero.[7]

$$\vec{F}_{ideal} \cdot \delta\vec{r} = 0$$

This arises when your constraint force is perpendicular to the virtual displacements your system allows. And while it may sound rare or incredibly specific, it happens way more frequently than you'd think.

- Imagine a hockey puck sliding over the ground. The primary constraint in this system is that the puck is lying still on the ice - no vertical motion. This is created by a constraint force, the normal force, acting perpendicular to the ice floor. Any allowed virtual displacement, that thus respects the constraints, is simply the puck sliding horizontally over the ice. In

other words, the virtual displacement will always be tangential to the ice floor. Because the force is now always perpendicular to the virtual displacement, the virtual work here is zero and this force is an ideal constraint force.

- Imagine the single pendulum example. The pendulum's rod has a fixed length, which is exactly the constraint. To assure this, there is a constraint force acting along the rod. The only virtual displacements that are allowed are rotational movements, so the movement vectors are tangential to the circle that the rod describes. Once again force and virtual displacement are perpendicular, causing an ideal constraint force with no virtual work.

2.2 D'Alembert's principle

D'Alembert's principle in dynamics makes use of ideal constraint forces to find a relation between the known, external forces and the time derivative of momentum.

Theorem 2.2.1 (D'Alembert's Principle). For a set of point masses under the influence of external forces $\vec{K}_i$ and internal forces, where the latter are ideal, it holds for some virtual displacements $\delta\vec{r}_i$ that[7]

$$\sum_{i=1}^{n} (\vec{K}_i - \dot{\vec{p}}_i) \cdot \delta\vec{r}_i = 0.$$

Proof. We start at Newton's second law. It states that the derivative of momentum for some particle is equal to all forces acting upon it. This includes both external (known) forces $\vec{K}_i$ and internal (constraint) forces $\vec{N}_i$,

$$\forall i : \vec{K}_i + \vec{N}_i = \dot{\vec{p}}_i.$$

Without loss of generality we can introduce a virtual displacement $\delta\vec{r}_i$ that respects all the system's constraints,

$$\forall i : (\vec{K}_i + \vec{N}_i - \dot{\vec{p}}_i) \cdot \delta\vec{r}_i = 0$$

The keen eye may notice that this momentum derivative could be regarded as an inertial force. Now we use the theorem's assumption that all constraint forces are ideal under allowed virtual displacements. This causes an entire term to disappear. Along with that, we can take the sum and thus conclude that

$$\sum_{i=1}^{n} (\vec{K}_i - \dot{\vec{p}}_i) \cdot \delta\vec{r}_i = 0.$$

□

D'Alembert's principle is the basis of Lagrangian mechanics. It can be used as one of multiple methods to derive its most important equation, namely $\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{q}_j}\right) - \frac{\partial L}{\partial q_j} = 0$. While we will not be using this approach here, what we're about to cover is completely equivalent. It results in the same physics, albeit more free, complicated and numerically solvable. Both principles can also be derived from each other.

2.3 Gauss's principle of least constraint

To understand Gauss's principle of least constraint, we must think about how constraints are actually incorporated into simulations.

The first step is always to think about known forces. Calculate those using their "famous" equations and sum their vectors to find the total external force acting upon each particle. If you just run the simulation now, no constraints are being taken into account: rods that should have a fixed length will stretch out forever, objects will fall through floors and walls will turn into thin air.

To satisfy constraints, we will obviously need to introduce the internal forces. After doing so, the actual acceleration (or total force) acting on the particle will have changed. The insight Gauss had is that technically speaking there are infinitely many possible ways to edit your acceleration in order to have constraints satisfied. Just imagine the single pendulum. The two ends of the rod - or every particle in the rod - feel internal forces that altogether satisfy the constraint, when combined with the external forces. But hypothetically you could make the internal force on one end a bit stronger, and just make the force on the other end an equal amount weaker. Same motion, different individual changes in acceleration.

So which option out of all of them should we pick to satisfy our constraints? Like with a lot of things in physics, it comes down to a variational formulation: we're going to minimize some quantity.

Theorem 2.3.1 (Gauss's Principle of Least Constraint). To correctly describe a system of particles under a given constraint, where m_i are their individual masses, $\vec{K}_i$ are the external forces, the quantity

$$Z = \sum_{i=1}^{n} m_i \left| \ddot{\vec{r}}_i - \frac{\vec{K}_i}{m_i} \right|^2$$

must be minimized to some final accelerations $\ddot{\vec{r}}_i$ under those same constraints.[14] [8]

Proof. We will derive the principle from D'Alembert's Principle. Recall the statement

$$\sum_{i=1}^{n} (\vec{K}_i - \dot{\vec{p}}_i) \cdot \delta\vec{r}_i = 0.$$

Next we assume that the particle masses remain constant, such that we can rewrite momentum and find

$$\sum_{i=1}^{n} (\vec{K}_i - m_i \ddot{\vec{r}}_i) \cdot \delta\vec{r}_i = 0.$$

These $\ddot{\vec{r}}_i$ are the resulting accelerations, so they can be split into effects of external forces, as well as a correction - the internal forces part - to satisfy constraints,

$$\ddot{\vec{r}}_i = \frac{\vec{K}_i}{m_i} + \vec{\alpha}_i.$$

We can also rewrite D'Alembert as

$$\sum_{i=1}^{n} m_i \left(\ddot{\vec{r}}_i - \frac{\vec{K}_i}{m_i} \right) \cdot \delta \vec{r}_i = 0$$

or equivalently with the above definition,

$$\sum_{i=1}^{n} m_i \vec{\alpha}_i \cdot \delta \vec{r}_i = 0. \tag{2.1}$$

This homogeneous equation can also be viewed from a variational perspective. The coordinates we are varying are the constraint-imposed corrections $\vec{\alpha}_i$ - also captured in the final accelerations. If we introduce

$$Z = \frac{1}{2} \sum_{i=1}^{n} m_i \left| \vec{\alpha}_i \right|^2$$

then a virtual change in Z can be written as

$$\delta Z = \sum_{i=1}^{n} m_i \vec{\alpha}_i \cdot \delta \vec{\alpha}_i.$$

Since virtual changes in constraint-induced corrections $\delta \vec{\alpha}_i$ are directly related to our virtual displacements $\delta \vec{r}_i$ - the constraints define which displacements are allowed - then from 2.1 it follows that $\delta Z = 0$. In other words, we must variationally tweak $\vec{\alpha}_i$ to find stable extreme values of Z - minimums. The $\frac{1}{2}$ term will not impact this, and we can substitute back our definition of $\vec{\alpha}_i$ to find that we need to solve for a minimum of the quantity

$$Z = \sum_{i=1}^{n} m_i \left| \ddot{\vec{r}}_i - \frac{\vec{K}_i}{m_i} \right|^2.$$

□

As mentioned previously, this new principle is in fact completely equivalent to D'Alembert's Principle. This should not be shocking, as we just derived one from the other. The difference is in how both of them describe the same mechanics. D'Alembert's Principle describes an equivalence-based solution. Through solving it in Lagrangian mechanics, one will use differential equations to describe the motion of an entire system. What we've done for Gauss' Principle of Least Constraint is convert it into an optimization problem. Now we can simply find the final accelerations $\ddot{\vec{r}}_i$ as those that minimize the quantity Z[1]. Furthermore, if we were to subtract the external accelerations, we can find the mechanically correct internal forces.

2.4 Lagrange multiplier for constrained optimization

Now that we've converted our search for the internal forces into an optimization problem - with the final accelerations as coordinates - we just need a mathematical tool to solve it. This is not a normal optimization problem, where we're going to calculate some partial derivatives and call

[1] Z stands for the German "Zwang", literally meaning "compulsion" or "constraint". This is the nomenclature Gauss himself used.

it a day. The minimum also has to respect the system's constraints.

We can once again draw the comparison to using D'Alembert's Principle. Since this is an equation, not an optimization problem, the constraints are now encoded in your coordinates. Specifically, remember the fact that when solving a system with Lagrangian mechanics, generalized coordinates must be chosen. That's because every (holonomic) constraint present in the system rules out one coordinate. With Gauss's Principle things are different: we're still using Cartesian coordinates and the constraints are simply extra requirements on top of the fact our force/acceleration result must minimize some quantity.

So how does one solve a constrained optimization problem? In other words, given a function $f(\vec{x})$, where $\vec{x}$ are all the coordinates, and a (holonomic) constraint on those coordinates, given by some $g(\vec{x}) = 0$, how do you find the $\vec{x}^*$ that both minimizes f and respects g?

Theorem 2.4.1 (Lagrange Multiplier Theorem). To solve a constrained optimization problem, where $f(\vec{x})$ is the function to minimize/maximize, $g(\vec{x}) = 0$ is the constraint, and λ is the Lagrange multiplier, it must hold that

$$\nabla f(\vec{x}) + \lambda \nabla g(\vec{x}) = \vec{0}$$

as well as $g(\vec{x}) = 0$.[15]

It is best to demonstrate this theorem with a simple calculus example, before we apply it to our mechanical problem.

Example 2.4.1. Assume $f(x, y) = xy$ and $g(x, y) = x^2 + y^2 - 1 = 0$. First let's calculate the necessary gradient vectors,

$$\nabla f(x, y) = \begin{pmatrix} y \\ x \end{pmatrix}$$

$$\nabla g(x, y) = \begin{pmatrix} 2x \\ 2y \end{pmatrix}.$$

Following the theorem, we then introduce this yet unknown Lagrange multiplier λ and say

$$\begin{pmatrix} y \\ x \end{pmatrix} + \lambda \begin{pmatrix} 2x \\ 2y \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}.$$

The trick now is to get a system of equations out of this, which we can use to eliminate λ and be left with a relation between x and y. Here we get

$$\begin{cases} y + 2\lambda x = 0 \\ x + 2\lambda y = 0 \end{cases}$$

from which it can be derived that either $x = y$ or $x = -y$. The final step is to enforce the constraint, so $g(x, y) = 0$ is the last thing left to be fulfilled. Performing these final steps

results in four possible optimal points,

$$\left(\frac{1}{\sqrt{2}},\frac{1}{\sqrt{2}}\right),\left(-\frac{1}{\sqrt{2}},\frac{1}{\sqrt{2}}\right),\left(\frac{1}{\sqrt{2}},-\frac{1}{\sqrt{2}}\right),\left(-\frac{1}{\sqrt{2}},-\frac{1}{\sqrt{2}}\right).$$

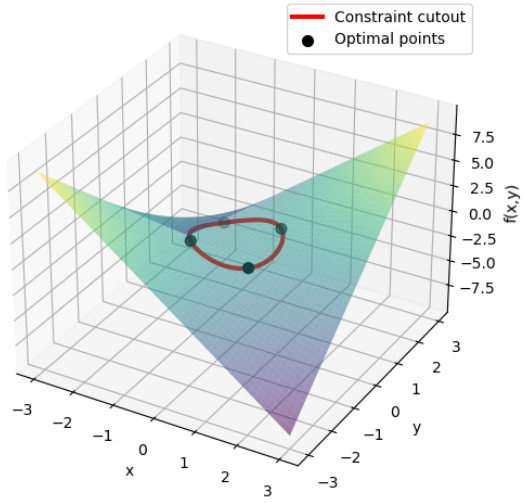

Figure 2.1: Visualization of example 2.4.1

Chapter 3

Constraints as linear algebra

3.1 Gauss's principle in matrix form

It is time to convert Gauss's principle and the aforementioned constraints into their linear algebra form. Since this is an important step, we'll take our time with it. First, let's recall the quantity Gauss defined to be minimized,

$$Z = \frac{1}{2}\sum_{i=1}^{n} m_i \left| \ddot{\vec{r}}_i - \frac{\vec{K}_i}{m_i} \right|^2 .^1$$

To express the masses in our system, we will require a block-diagonal mass matrix M. In two dimensions it can be defined as

$$M = \mathrm{diag}(m_1 I_2, m_2 I_2, \ldots) = \begin{pmatrix} m_1 & 0 & 0 & 0 & \cdots \\ 0 & m_1 & 0 & 0 & \cdots \\ 0 & 0 & m_2 & 0 & \cdots \\ 0 & 0 & 0 & m_2 & \cdots \\ \vdots & \vdots & \vdots & \vdots & \ddots \end{pmatrix}$$

which is a matrix with dimensions $2N \times 2N$.

Next is the actual acceleration. Both the final acceleration - the thing we're looking for - and the initial acceleration due to known forces - equivalent to $\frac{\vec{K}}{m}$ - will be written as column vectors. In two dimensions this becomes

$$\vec{a} = \begin{pmatrix} \vec{a}_1 \\ \vec{a}_2 \\ \vdots \end{pmatrix} = \begin{pmatrix} a_{1x} \\ a_{1y} \\ a_{2x} \\ a_{2y} \\ \vdots \end{pmatrix}$$

which is a vector of length 2N. Also note that for convenience purposes we will stop drawing the vector arrow in notation from now on. We can then redefine Gauss' quantity as

[1] The 1/2 factor was not present before. It is a standard in this formulation, so we will use it. Note however that the minimalization is not impacted by a constant factor.

$$Z = \frac{1}{2}(a - a_0)^T M(a - a_0).$$

The two terms on the right form another column vector of length 2N, that being $a - a_0$ but each term correctly multiplied by the according mass. Adding on the leftmost vector multiplication, it's the transposition that makes this turn into a scalar at the end. That scalar being the exact same thing we defined in Gauss's principle earlier.

3.2 Constraints in matrix form

Next we need to convert our constraints to a matrix form. Suppose we have k unique constraints, given by

$$\begin{cases} \phi_1(\vec{r}_1, \vec{r}_2, \ldots) = 0 \\ \phi_2(\vec{r}_1, \vec{r}_2, \ldots) = 0 \\ \qquad\vdots \\ \phi_k(\vec{r}_1, \vec{r}_2, \ldots) = 0 \end{cases}.$$

We already know that differentiating these twice results in the proper constraint equations involving acceleration terms - or if we're performing numerical simulations we also do Baumgarte stabilization. In the end this will result in k equations with a total of 2N variables - acceleration in both directions per particle. The entirety of the constraint problem will be summarizable via

$$Aa = b$$

where A is a matrix of dimensions $k \times 2N$ and b is a column vector of length k.

Example 3.2.1. To demonstrate, let's assume two particles are connected by a massless rod of some fixed length L. This will be covered as one of the examples in chapter 5. The eventual expression we get for the constraint is

$$C(\vec{a}_1, \vec{a}_2) = (\vec{r}_2 - \vec{r}_1) \cdot (\vec{a}_2 - \vec{a}_1) + (\vec{v}_2 - \vec{v}_1)^2 = 0.$$

This is a single constraint, so $k = 1$, but with two particles our position, velocity, and acceleration vectors each have a length of 4. It might not be fully obvious what the correct matrix A is, since vectors are involved, so let's write this one out:

$$(x_2 - x_1)(a_{2x} - a_{1x}) + (y_2 - y_1)(a_{y2} - a_{y1}) = -(\vec{v}_2 - \vec{v}_1)^2$$

Now it should be more obvious, our matrix is equal to

$$A = \begin{bmatrix} -(x_2 - x_1) & -(y_2 - y_1) & (x_2 - x_1) & (y_2 - y_1) \end{bmatrix} = \begin{bmatrix} -(r_2 - r_1)^T & (r_2 - r_1)^T \end{bmatrix}$$

and for the vector - which in this case is just a scalar - we find

$$b = -(\vec{v}_2 - \vec{v}_1)^2.$$

3.3 Lagrange multipliers for multiple constraints

As before, the next step is to solve a constrained optimization problem. The coordinates of the problem are all those final accelerations, or in linear algebra terms simply the vector a. We covered the basic Lagrange multiplier theorem for one constraint, but it can be easily expanded to multiple constraints. We simply introduce an equal amount of Lagrange multipliers, one per each of the k constraints.

Previously the general problem with one constraint and n coordinates - n is the length of $\vec{x}$ - could be solved by working out the system

$$\begin{cases} \nabla f(\vec{x}) + \lambda \nabla g(\vec{x}) = \vec{0} \\ g(\vec{x}) = 0 \end{cases}$$

which involves $n+1$ coordinates - the entirety of $\vec{x}$ plus λ - and a total of $n+1$ equations - the gradients create n and then one for the constraint.

Theorem 3.3.1 (General Lagrange Multiplier Theorem). To solve a constrained optimization problem, where $f(\vec{x})$ is the function to minimize/maximize, $\{g_1(\vec{x}), \ldots, g_k(\vec{x})\}$ is the set of constraints to satisfy, and $\vec{\lambda}$ is the vector of length k containing the Lagrange multipliers, it must hold that

$$\begin{cases} \nabla f(\vec{x}) - \lambda_1 \nabla g_1(\vec{x}) - \ldots - \lambda_k \nabla g_k(\vec{x}) = \vec{0} \\ g_1(\vec{x}) = 0 \\ \vdots \\ g_k(\vec{x}) = 0 \end{cases}$$

[15]

Once again, this is a system with $n+k$ unknowns and $n+k$ equations.

3.4 KKT matrix

In our specific case, the coordinates are all the elements of the vector a, and the constraint is given by $Aa - b = 0$, so one equation for each of the k rows. Due to the shape of these constraints, it shouldn't be a surprise that the gradients we're looking for are just the rows of A. Take for example the first constraint, which corresponds to the first row of A and first element of b. We can write it plainly as $A_{1,1}a_{1x} + A_{1,2}a_{1y} + \ldots + A_{1,2N}a_{ny} - b_1 = 0$, now taking the gradient of this simply gives us $[A_{1,1}\ A_{1,2}\ \ldots\ A_{1,2N}]$, which is indeed the first row of the matrix A. The gradient of the function to minimize, which is $Z = \frac{1}{2}(a - a_0)^T M(a - a_0)$, is given by $M(a - a_0)$. Therefore we must solve the system

$$\begin{cases} M(a - a_0) - A^T\lambda = 0 \\ Aa = b \end{cases}$$

by first finding that

$$a = a_0 + M^{-1}A^T\lambda$$

and plugging into the second equation to find

$$A(a_0 + M^{-1}A^T\lambda) = b$$

$$(AM^{-1}A^T)\lambda = b - Aa_0.$$

For our case, the system can also be written as

$$\begin{bmatrix} M & -A^T \\ A & 0 \end{bmatrix} \begin{bmatrix} a \\ \lambda \end{bmatrix} = \begin{bmatrix} Ma_0 \\ b \end{bmatrix}$$

in which the matrix is known as the Karush-Kuhn-Tucker matrix. Matrices like these - symmetrical, positive and often diagonally dominant - always show up when solving constrained optimization problems in multiple coordinates.

Chapter 4

Numerical constraint solvers

If we continue directly from the previous chapter, we now need to solve the linear system given by

$$(AM^{-1}A^T)\lambda = b - Aa_0$$

which results in a solution for λ that can then lead to the eventual final accelerations via

$$a = a_0 + M^{-1}A^T\lambda.$$

4.1 Direct VS iterative solving

The weigh-off needs to be made between direct solving methods and iterative solving methods.

Direct solving methods, like Gauss elimination combined with backward substitution, will give an exact solution up to errors caused by numerical stability and rounding.

Iterative solving methods start with a guess for the solution and use a fixed iteration technique to inch closer and closer to the exact solution, only stopping once they're close enough. Though it sounds worse than just solving directly, these methods are often much faster in their computational complexity and can be tweaked to find the right balance between computing time and accuracy. Furthermore, methods like impulse-based solving incorporate the time-stepping part of the simulation directly into the constraint solving. This highlights why iteration isn't so bad after all: isn't taking steps in time exactly that?

4.2 Gauss-Seidel method

The Gauss-Seidel method is an iterative method that primarily focuses on solving systems of non-linear equations. In other words, it strives to find the vector $\vec{x}$ of length n for which all functions $f_1(\vec{x}), \ldots, f_n(\vec{x})$ are equal to zero. The way it works is simple: after having chosen a starting point for the iteration, we first approximate the vector function using the Jacobian - a

sort of higher dimension Taylor series around the current iteration point $\vec{x}^{(k)}$.

$$\vec{y}(\vec{x}) = \vec{f}(\vec{x}^{(k)}) + \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \frac{\partial f_n}{\partial x_2} & \cdots & \frac{\partial f_n}{\partial x_n} \end{pmatrix} (\vec{x} - \vec{x}^{(k)})$$

This large matrix of partial derivatives, all of which should be evaluated in $\vec{x}^{(k)}$, is called the Jacobian and will be referred to as J or $J^{(k)}$ from now on. The actual iteration step says that our next point should be the one where $\vec{y}(\vec{x}) = 0$. In the one-dimensional case, you can imagine this function to be the tangential line to f at $x^{(k)}$, and the intersection between this line and the x axis is your new point. Thus we find the expression

$$J(\vec{x}^{(k+1)} - \vec{x}^{(k)}) = -\vec{f}(\vec{x}^{(k)}).$$

What we've done so far is called the Newton-Raphson method. If you're performing that algorithm, you'd stop here and find the next point $\vec{x}^{(k+1)}$ by solving this linear system. But this doesn't help us! We're trying to use this technique to solve an actual linear system iteratively, and if we were to plug it into this, it would just force us to use (slow) Gauss elimination - a direct method! So we need to use an approximated version of this technique. It involves using only the diagonal of your Jacobian and setting all the other elements to zero.

In this specific case, you can rewrite the step as

$$\vec{x}^{(k+1)} = \vec{x}^{(k)} - J^{-1}\vec{f}(\vec{x}^{(k)})$$

and since J is now diagonal - the inverse literally being the same with each element inverted - you can perform the step per coordinate as

$$x_i^{(k+1)} = x_i^{(k)} - \frac{f_i(\vec{x}^{(k)})}{\frac{\partial f_i}{\partial x_i}(\vec{x}^{(k)})}.$$

An important thing to note is that the order in which you number the different functions f_i determines which partial derivatives will land on the diagonal. While it may seem unimportant, a different set of partial derivatives may lead to convergence becoming divergence or vice versa. But the real Gauss-Seidel method goes even further - this is just the Jacobi method. The above equation is performing the step for the i-th coordinate of $\vec{x}$, so we have already stepped coordinates $1, \ldots, i-1$. Why not already use those new coordinates in the function and derivative calls, like below?

$$x_i^{(k+1)} = x_i^{(k)} - \frac{f_i(x_1^{(k+1)}, \ldots, x_{i-1}^{(k+1)}, x_i^{(k)}, \ldots, x_n^{(k)})}{\frac{\partial f_i}{\partial x_i}(x_1^{(k+1)}, \ldots, x_{i-1}^{(k+1)}, x_i^{(k)}, \ldots, x_n^{(k)})}$$

Doing this will greatly increase the speed of convergence, since new information is being used more rapidly. On the other hand, we've introduced yet another annoying degree of freedom: the order in which to step the coordinates. If you were just doing Jacobi iteration, this wouldn't matter, since you keep using the old coordinates to evaluate. But since you're using the coordinates you've already stepped to calculate the next one, the order now matters! A different order can imply a whole different level of accuracy or straight-up divergence. This however is not as

fragile as the function ordering from before.

Now that the method has been covered, let's apply it to our linear system.

$$(AM^{-1}A^T)\lambda = Aa_0 - b$$

For the sake of readability, let's define $S = AM^{-1}A^T$ and $r = Aa_0 - b$. This reduces the problem to solving $S\lambda = r$, where S is a matrix of dimensions $k \times k$ and r is a vector of size k. The reason why we can even use Gauss-Seidel on linear systems is because $S\lambda = r$ is the same as $S\lambda - r = 0$, which defines a linear systems, or k functions that should be zero at our λ. The Jacobian here is literally just S itself, which makes things a lot simpler. We get

$$\lambda_i^{(k+1)} = \lambda_i^{(k)} - \frac{\sum_{j=1}^{i-1} S_{ij}\lambda_j^{(k+1)} + \sum_{j=i}^{k} S_{ij}\lambda_j^{(k)} - r_i}{S_{ii}}$$

4.3 Impulse-based simulations

The next step is to completely get rid of accelerations and perform our system solving on a velocity level, such that we can incorporate it directly into the time steps.

Like we did Verlet-style, we update velocities via

$$v^{n+1} = v^n + \Delta t a.$$

We know what a is from solving multi-dimensional Lagrange multipliers, so we get

$$v^{n+1} = v^n + \Delta t a_0 + \Delta t M^{-1}A^T\lambda.$$

Now we'll define some terminology to shorten things, like

$$v_0^{n+1} = v^n + \Delta t a_0$$

which is called the unconstrained velocity, just like a_0 was the unconstrained acceleration - only including those known or external forces. We also define the core of this section, the constraint impulse J - not the Jacobian! - as $J = \Delta t\lambda$. It's quite literally a jump from unknowns in the acceleration domain to unknowns in the velocity domain. We end up with

$$v^{n+1} = v_0^{n+1} + M^{-1}A^T J.$$

Using capital J as a symbol might make the constraint impulse seem like a matrix, but it is of course still a vector of length k.

What's most important to see now is that if we have our simple ideal, holonomic constraints, they will be expressed - or expressable - at a velocity level via

$$Av = 0$$

where v is the long vector of length 2N containing the velocities $v_{1x}, v_{1y}, v_{2x}, \ldots$ and A is actually the same matrix as we had for acceleration-based constraints. This makes sense, as deriving $Av = 0$ once more to get to the acceleration level would yield two terms due to the product formula: Aa and b, where b is created by the derivatives of the coefficients that were next to

the elements in v. Take for example the rigid rod example from before. At a velocity level it is expressed as

$$2(\vec{r}_2 - \vec{r}_1) \cdot (\vec{v}_2 - \vec{v}_1) = 0$$

such that the matrix A (1×4) would once again become

$$A = \begin{bmatrix} -(x_2 - x_1) & -(y_2 - y_1) & (x_2 - x_1) & (y_2 - y_1) \end{bmatrix} = \begin{bmatrix} -(r_2 - r_1)^T & (r_2 - r_1)^T \end{bmatrix}.$$

Taking the derivative again yields both Aa but also includes the derivative of $\vec{r}_2 - \vec{r}_1$, which forms the b vector we saw at the acceleration level.

So at each time step, the new velocity - which has been expressed according to Gauss's principle and a so far unknown J - must respect these velocity-level constraints, meaning

$$A\left(v_0^{n+1} + M^{-1}A^T J\right) = 0$$

$$(AM^{-1}A^T)J = -Av_0^{n+1}$$

which is the equation to solve for J, preferably using Gauss-Seidel. So to recap, here's how we got here:

- Gauss's principle tells us how constraints should minimally affect the unconstrained acceleration.
- Using multi-dimensional Lagrange multipliers, we were able to find the expression for the constrained acceleration, which contains an unknown λ found by respecting the constraint $Aa = b$. This equation follows directly from the fact we're minimizing Gauss's quantity Z, so using it anywhere satisfies the principle.
- If we were to stop here, we would use Gauss-Seidel to solve for λ, calculate the constrained acceleration, and apply it for a velocity time step.
- But we can make this faster by skipping the acceleration calculation. Instead, we use our previous expression for the constrained acceleration to directly define what v^{n+1} would be, and use velocity-expressed constraints to solve for J - which is just λ in a different suit.
- This not only skips some steps, but also slightly increases the efficiency of the Gauss-Seidel solve, since there is no more b in our constraint expression.

4.4 Projected Gauss-Seidel (PGS)

The final step in working out the core of mechanical simulations is to mix Gauss-Seidel directly with incremental fixes in the velocity. The grand scheme is something like this, performed each time-step:

- Calculate the unconstrained velocities, by using the previous velocity and the unconstrained acceleration from external forces like gravity, springs, ...
- Initiate the constraint impulse - our unknown - as $J = 0$.
- Perform Gauss-Seidel, meaning we do multiple full updates of J.
- Within each GS iteration, we're updating every J_i one by one - and using the new items ($j < i$) instantly.

- We can rewrite GS to tell us what ΔJ_i will be, so that we can update J but also instantly update the velocity v with only the impact from J_i changing. We do this because our rewritten version of GS will refer to the (constrained) velocity itself in calculation - so the next J_{i+1} needs the new, more correct version.
- When we have performed enough full GS iterations, each one itself consisting of an update to each J_i along with an update to v, we should have a proper numerical estimation of the constrained velocity.
- Finally just use $x^{(i+1)} = x^{(i)} + v\Delta t$ to take one step in time for positions.
- Continue to the next time-step.

Before we get into the mathematical details, it's important to note how doing it this way makes much more sense when viewed from the constraint perspective. As you'll see, the calculation for ΔJ_i and equivalently the immediate update for v, will only involve row i of our matrix A (A_i), which corresponds exactly to one constraint in our system: the one that J_i is linked to. So we won't construct the whole matrix anymore, it makes more sense now to reason from constraint to constraint - row to row, J_i to J_{i+1} - and is also much more efficient.

Let's derive the actual proper calculations. Begin by remembering what we're working with, which is

$$v^{n+1} = v_0^{n+1} + M^{-1}A^T J$$
$$(AM^{-1}A^T)J = -Av_0^{n+1}.$$

If we perform Gauss-Seidel for this latter system, we are saying

$$J_i^{(new)} = J_i^{(old)} - \frac{1}{S_{ii}}\left(\sum_{j=1}^{k} S_{ij}J_j - r_i\right).$$

Here it's important to note that the sum is no longer split up into the $j < i$ and $j \geqslant i$ parts. This is because we're instantly updating the J vector, so simply referencing J_{i-1} already gives the updated value - from the previous step - and further elements are the old ones, yet to be overwritten. Now let's take a closer look at S and r in this case. We can see that

$$S_{ij} = A_i M^{-1} A_j^T$$

where A_i is row i of the matrix A. The rightmost term is equivalent to column j of A^T, which is indeed just the transpose of row j of the matrix A. It is trivial to check that given this form, S_{ii} in the denominator will always be non-zero. We also know

$$r_i = -A_i v_0^{n+1}$$

such that the update becomes

$$J_i^{(new)} = J_i^{(old)} - \frac{A_i v_0^{n+1} + A_i M^{-1} \sum_{j=1}^{k} A_j^T J_j}{A_i M^{-1} A_i^T}. \tag{4.1}$$

Now let's take a closer look at the constrained velocity expression and see how we can introduce this term into our constraint impulse update, such that the velocity can be iterated too. Note that the equivalence

$$A^T J = \sum_{j=1}^{k} A_j^T J_j$$

holds and thus we can say

$$M^{-1}\sum_{j=1}^{k} A_j^T J_j = v^{n+1} - v_0^{n+1}.$$

Plugging into eq. (4.1) then gives

$$J_i^{(new)} = J_i^{(old)} - \frac{A_i v^{n+1}}{A_i M^{-1} A_i^T}$$

or equivalently - and more usefully -

$$\Delta J_i = -\frac{A_i v^{n+1}}{A_i M^{-1} A_i^T}. \tag{4.2}$$

This seemingly simple fraction gives us the change in J_i for this step of this iteration of Gauss-Seidel. We then use it to update the J vector and also to update our constrained velocity. The latter can be done via

$$v^{n+1} = v_0^{n+1} + M^{-1}\sum_{j=1}^{k} A_j^T J_j$$

where if only one coordinate J_i changes, the influence on v^{n+1} is

$$\Delta v^{n+1} = M^{-1} A_i^T \Delta J_i.$$

If we wish to further increase the efficiency of the technique, we could try to warm-start our J vector. Right now we reset $J = 0$ at each time-step, before starting the GS iterations. But if the time-steps are small enough, we can expect minimal changes to J each time, so it'd be useful to warm-start $J_0^{(k+1)} = J^{(k)}$ and let it do the few remaining iterative changes from there. This causes faster convergence and saves on computation. We do have to be wary about the velocity here. If you say $J_0 = 0$, then initiating $v^{n+1} = v_0^{n+1}$ - the unconstrained velocity - and applying $\Delta v^{n+1} = M^{-1} A_i^T \Delta J_i$ per update works perfectly. But if you start with a non-zero J, you also can't start with the velocity being just the unconstrained velocity, but rather $v^{n+1} = v_0^{n+1} + M^{-1} A^T J$ and only then can you start applying the correct tiny updates!

Warm-starting comes with its downsides though. It's a great method to increase efficiency by remembering the past, but what happens if sudden changes in the system occur? Say you have two balls stacked on top of each other and you drop a huge third ball on top of them. The system will stabilize and converge thanks to warm-starting - even if your amount of iterations is small - but if you now remove the third huge ball, you'll notice the top ball bouncing up. This is because in the first frame where the huge ball is gone, we warm-start J using the one from the previous frame, when the ball was still present. And if your amount of iterations is small, there won't be enough opportunity for J to adapt to the new situation, so inconsistencies like the bouncing occur.[4]

4.5 Incorporating contacts

All the constraints we've covered so far are holonomic, because they can be expressed as an analytical relation, like $Av = 0$. This however will not always be the case, especially when

dealing with contacts.

Contact is a posh name for what will basically be collisions between objects in the simulation. While this might only make you imagine two balls smashing into each other, technically an object sliding on a floor or bouncing on a stationary wall is also considered a contact. Contacts are modeled via inequalities, such as $Av > 0$. But unlike permanent connections or holonomic constraints between particles, these inequality constraints are introduced in a different way. The contact constraint between two objects should only be added to the list of constraints - equivalently the A matrix - if those objects are actually colliding. The system that detects these overlaps is completely seperate from the dynamical solver, and we will cover it later. For now, just imagine that at the start of each time-step, some magical function is able to tell us which particles are colliding.

For each pair of colliding particles - or objects - one contact constraint will be added, or one row in A. If we imagine this being a collision between object i and object j, whose velocities appear in v at index $2i$ respectively $2j$, we would add this row:

$$\begin{bmatrix} 0 & \dots & n_x & n_y & \dots & -n_x & -n_y & \dots & 0 \end{bmatrix}, \quad n_x \text{ at position } 2i,\ -n_x \text{ at } 2j$$

Where $\vec{n} = (n_x, n_y)$ is the normal vector pointing from particle j to particle i. Now let's see what calculating $A_k v$ - k representing this row - would equate to.

$$A_k v = \vec{n} \cdot \vec{v}_i - \vec{n} \cdot \vec{v}_j = \vec{n} \cdot (\vec{v}_i - \vec{v}_j)$$

This is a relative velocity projected in the normal direction. More specifically, the right-hand term represents the velocity of i from the perspective of j. So if this expression is positive, then from object j's perspective, object i is moving in the direction of $\vec{n}$, so away from j. For objects that are colliding, we want to constrain their velocities such that they move apart relatively, so clearly what we need to enforce with this constraint is

$$A_k v > 0.$$

Mathematically, both the equality and inequality constraints we've covered can generally be called KKT conditions[16]. For equality constraints the solving method is equivalent to what we covered with Lagrange multipliers. For inequality constraints, we also introduce Lagrange multipliers, but the conditions they must satisfy are slightly different. The same update from eq. (4.2) applies, and the velocity still changes depending on how J_k changes, but we also enforce

$$\begin{cases} J_k > 0 \\ A_k v \cdot J_k = 0 \end{cases}.$$

The first statement is an obvious one. If you look at the velocity update equation

$$v^{n+1} = v_0^{n+1} + M^{-1} \sum_{j=1}^{k} A_j^T J_j$$

you'll see that if J_k is positive, the velocity increase - basically a force - on object i is in the direction of $\vec{n}$, so away from object j. In other words, enforcing $J_k > 0$ assures the created constraint force - which could be like a normal force - is always pushing objects apart.
As for the second statement, let's look at two main situations to observe why this is true.

- Option 1: $A_k v_0 > 0$

  This means that at the start of our iterations, even though the objects are colliding, they are already moving apart and thus satisfying the constraint. According to eq. (4.2) this will cause J_k to decrease and decrease, until it reaches zero where it is clamped because of the $J_k > 0$ requirement. In the end, this means the constraint has no effect, does not attribute to a velocity change and thus no forces - we call the constraint inactive. Since this situation has pushed to $J_k = 0$, the second statement does indeed hold.

- Option 2: $A_k v_0 < 0$

  This means that at the start of our iterations, the objects are trying to penetrate each other even more, since their relative velocities do not satisfy the "moving apart" constraint. According to eq. (4.2) this will cause J_k to increase and increase, applying a pushing force to the particles, until the constraint is minimally satisfied, or $A_k v = 0$. At that point, ΔJ_k will also be 0 and since there will be no more changes, the second statement does indeed hold.

So in practice we can implement contacts as follows:

- Use an external method at the start of a time-step to list which pairs of objects are colliding.
- For each of these pairs, an inequality constraint is added to the system that time-step.
- When encountering these constraints in the iteration, calculate and apply ΔJ_k as always. However do not apply it to the velocity just yet.
- First clamp J_k to be positive, often by saying $J_k = \max(0, J_k + \Delta J_k)$.
- Calculate the actual change $\Delta J_{k,real} = J_{k,old} - J_{k,new}$ which can differ from ΔJ_k if clamping was performed. For example, if J_k = 3 and $\Delta J_k = -5$, then the resulting $J_k = 0$ and thus $\Delta J_{k,real} = -3$.
- Apply the velocity update using this actual change, so $\Delta v^{n+1} = M^{-1} A_k^T \Delta J_{k,real}$.

Some might still be wary of this implementation, since the way we actually implemented the new constraint, with the J_k updates, might make it look like we're still enforcing $Av = 0$ instead of $Av > 0$. You're right, theoretically we are enforcing the equality. But we also clamp $J_k > 0$, which limits which forces can be used to make the constraint respected. In this case it only allows pushing forces, so the case where $Av < 0$ will be repaired until $Av = 0$, but if you already have $Av > 0$ then technically speaking it would do ΔJ_k until eventually $J_k < 0$ such that it could start working towards $Av = 0$. However we clamp J_k to be positive, so in this case it actually just stays at 0 and leaves the already satisfied constraint alone. It's this equality constraint combined with a clamp on the constraint impulse that lets us control the implied forces, which will become more evident with friction.

4.6 Incorporating friction

Friction between two objects happens if they have tangential relative velocity. It acts up to a certain maximum to reduce this velocity - this determines the direction. Friction happens between any two objects in contact, so it can be added alongside the actual contact constraint. This time, we will try to enforce the constraint saying 'no relative tangential velocity' using a new row in A. However, we will clamp J_k to some interval, such that the force trying to reduce

the tangential relative motion has a maximum, just like real friction. This interval's size should also depend on the normal force, caused by the constraint we covered in the previous chapter, so it is controlled by the eventual - or iterated - J_k for that constraint.

First let's take a look at the new row we need to introduce into A. As we just covered it should represent the relative tangential velocity as Av. So the row is

$$\begin{bmatrix} 0 & \dots & t_x & t_y & \dots & -t_x & -t_y & \dots & 0 \end{bmatrix}, \quad t_x \text{ at position } 2i,\ -t_x \text{ at } 2j$$

where $\vec{t} = (t_x, t_y)$ is the tangent vector, so the vector perpendicular to $\vec{n}$ from before. Technically speaking, on a 2D grid, there are two possibilities for such a vector. But it does not matter which one you use, the sign of Av and J_k will flip accordingly, always causing an opposing force. So now $A_k v = \vec{t} \cdot (\vec{v}_i - \vec{v}_j)$, the velocity of i from j's perspective, projected on the tangent vector.

Enforcing $A_k v = 0$ means the system we already have with J_k updating and gradually creating a force that changes v, will try to get this relative velocity down to zero. But friction has a maximum. J_k can be either positive or negative, whichever is needed to reduce the relative motion, but its magnitude should be fixed. In other words, we have to clamp

$$J_k \in [-\mu |J_{normal}|, \mu |J_{normal}|]$$

where J_{normal} is the constraint impulse that corresponds to the contact constraint for these same two colliding objects. This will guarantee that the highest possible absolute effect on velocity is $\Delta\vec{v}_i = \frac{\vec{t}\mu|J_{normal}|}{m_i}$, which is indeed equivalent to the way friction is usually calculated: some multiple of the normal force (in magnitude).

So once again this is introduced into the loop by calculating ΔJ_k as normal, updating J_k with it, clamping it to this interval, then calculating the real change and applying that to the velocity update. Since the iterations of J_k and J_{normal} are happening side-by-side, this gives rise to more complex interactions between constraints.

The keen-eyed among you might be terribly upset right now, because this system clearly doesn't distinguish between static and dynamic friction. You are right. Incorporating this differentiation would require you to take a look at the beginning $A_k v$, and testing if the maximum static friction - something like $J_k = \pm\mu_{static}|J_{normal}|$ - is sufficient to completely bring $A_k v$ down to zero. If it is not, then you know relative motion will still occur and thus you should switch to dynamic friction instead, where the dynamic μ is often smaller than the static one. However this requires you to already know the final iterated J_{normal}, and the proper transitioning from static to dynamic calculations - without stuff blowing out of proportions - is extremely hard to control. This is why real simulations do not make a distinction between the two types and just take a μ that averages the two. And to be honest, barely anybody would be able to tell the difference between the implementations.

4.7 Baumgarte stabilization

Let me introduce you to a shocking truth: as beautiful as these numerical iteration methods are, they remain numerical and thus fundamentally introduce errors into the system. Another truth is that our current iterations focus on velocity-based constraints. Take for example the fixed distance constraint, where we shove a rod of some constant length L between two particles or

objects. To implement this, we'd use the velocity-based constraint, $2(\vec{r}_2 - \vec{r}_1) \cdot (\vec{v}_2 - \vec{v}_1) = 0$. This way, the system will try to orient velocities using constraint forces such that the connected objects don't lengthen or shorten the distance between each other.

But this won't go without some numerical errors, either due to rounding or due to a limited amount of Gauss-Seidel iterations. These errors accumulate throughout the simulation and can cause the length of our rod to lose track of its original L. And since this velocity-based constraint carries no information about that original length, the system has no idea something is going wrong. The rod acts sort of like a grandma with dementia: she does her best to keep all the money in her purse, but she can never remember what the exact amount she should have is, and she fails to notice thieves stealing nickles and dimes from her every Δt.

The solution is to keep everything as it is now, apart from the solving equation for J. This equation arose from combining the expression for v with $Av = 0$. So now we'll instead define

$$Av = -\frac{\beta}{\Delta t} C(x).$$

[1] This makes the constraint act as more of a spring-dampened system, which now includes information about the positions x and some constraint C on them, telling the system to recoil back. We call $\frac{\beta}{\Delta t} C(x)$ the bias. The constraint we're trying to enforce on the positions here is $C(x) = 0$. So for the rod example the corresponding $C_k(x)$ would be $(\vec{r}_i - \vec{r}_j)^2 - L^2$, zero if the distance is properly conserved.

For the contact constraint, you might enforce there to be no penetration, so $C_k(x)$ represents the gap between the objects along the $\vec{n}$ direction. If it is negative, they are overlapping, if it is positive, they are already apart. But here you must once again make a distinction! We're going to try and enforce $C_k(x) = 0$, but if it is already positive then they're already apart and we need not enforce anything. So in reality you would say the bias is equal to $\frac{\beta}{\Delta t} \min(0, C(x))$.

For the friction constraint, you don't need to enforce anything for positions. All this constraint does is implement friction by reducing relative tangential velocity up to some maximum. No positional information is involved, so here $C_k(x) = 0$ at all times.

With this new expression we need to re-derive the solving system for J, as well as the Gauss-Seidel update equation. Luckily barely anything actually changes. We still arrive at

$$v^{n+1} = v_0^{n+1} + M^{-1} A^T J$$

and now use

$$Av = -\frac{\beta}{\Delta t} C(x)$$

to find the equation for J as

$$\left(AM^{-1}A^T\right) J = -Av_0^{n+1} + D$$

where we've defined

$$D = -\frac{\beta}{\Delta t} C(x).$$

The same steps as before also apply to implementing a Gauss-Seidel update. We once again fill in the values and through some substitutions find the update expression that involves the

current iteration of the constrained velocity. Only now we also have D in there, as

$$\Delta J_i = -\frac{A_i v^{n+1} - D_i}{A_i M^{-1} A_i^T}.$$

We can rewrite this to its final form

$$\Delta J_i = -\frac{A_i v^{n+1} + \frac{\beta}{\Delta t} C_i(x)}{A_i M^{-1} A_i^T}$$

and remember the same velocity update still holds,

$$\Delta v^{n+1} = M^{-1} A_i^T \Delta J_i.$$

4.8 Nonlinear Gauss-Seidel (NGS)

The solver as described up until this point would be perfectly capable of yielding good-looking, stable simulations. Its accuracy towards constraints however, is less desirable. If you run a distance constraint test simulation with Baumgarte stabilization, you'll notice the positional constraint it is supposed to enforce, $d - L = 0$, isn't zero but a small decimal value depending on your settings. The problem here is that the enforcing of Baumgarte stabilization, which right now is the only thing actually reminding the system of its positional accuracy, is mixed with updating an already informed velocity. We start with a velocity that is carried over from the last frame and already has external forces applied to it. Then the velocity-based constraint has to fight with the Baumgarte term in the numerator over ΔJ. Eventually it settles and the constraint is satisfied, but we pollute the next frame with positional corrections. A better approach would be to separate the positional constraint from the velocity one.[5]

So we'll remove the Baumgarte term in the numerator of our velocity iteration, and run that like before. This yields an updated (final) v^{n+1}. Before we apply that velocity for a positional step in time, we let a separate, but similar positional solver do its job to correct positional violations of the constraint. It uses a pseudo-velocity v^* - stored per body just like normal velocities and also including rotational velocity, initiated at zero such that it carries no conflicting information. Using the same Gauss-Seidel updates, but including the Baumgarte term in the numerator and being time-independent (set $\Delta t = 1$ everywhere), we can calculate ΔJ and find out what tiny velocity might be able to resolve the positional violation. We still perform this multiple times, say N, and apply the eventual velocity result alongside the normal one. While the remaining positional error will never be zero - simulations always remain numerical - they will be orders of magnitude smaller and the simulation will be more stable. Tinkering with the amount of iterations (N), the positional constraint stiffness (β) and the time-step detail (Δt) allows us to minimize the error even more, at the cost of computation and - for extreme values - stability. We will talk more about these hyperparameters later.

4.9 Further optimizations

In practice, more advanced techniques or completely different approaches can be used for specific applications to yield results with less numerical error and more stability. We will call it quits here, but the interested reader can always find more information via the documentation of popular physics engines such as Box2D and Bullet2D.

Chapter 5

Implementing rotation

So far we have completely neglected the possibility of rotation in our simulation. The objects we currently have are just controlled by translational movements, seen from the mass center - it's easy to mathematically prove that forces, speeds, and momentum can all be regarded from the mass center. But rotations are necessary, so we'll incorporate them.[6]

5.1 New state vector

Since we're looking at 2D motion, rotation only adds one degree of freedom. If you imagine a circle on the screen right now, it's rotational axis would be going in or out of your screen, along some magical axis not on our 2D grid. We will update the state vector to include an angle of rotation θ. How this angle is defined does not matter, since any constant difference between two implementations has no impact on the speed, their derivatives.

$$\vec{x}(t) = \begin{pmatrix} x_1(t) \\ y_1(t) \\ \theta_1(t) \\ x_2(t) \\ y_2(t) \\ \theta_2(t) \\ \vdots \end{pmatrix}$$

5.2 Gauss's principle with rotation

Gauss's principle also needs to be updated to deal with rotation. Once again its premise will be that any acceleration changes caused by constraints should be minimally invasive. For translations, these accelerations (squared) were weighted by mass. Well as you might expect for rotations, they're weighted by moment of inertia, I. These will thus need to be determined for every single object. But for simple objects like circles and squares, this is no big deal. In general, this contribution is written as

$$Z = \frac{1}{2}\left((a - a_0)^T M (a - a_0) + \sum_{i=1}^{n} (\alpha_i - \alpha_{i0})^T I_i (\alpha_i - \alpha_{i0})\right) \tag{5.1}$$

where α is the rotational acceleration - the second derivative of θ - and I_i is the inertia tensor. The latter is generally a symmetrical 3×3 matrix containing

$$\begin{pmatrix} I_{xx} & I_{xy} & I_{xz} \\ I_{yx} & I_{yy} & I_{yz} \\ I_{zx} & I_{zy} & I_{zz} \end{pmatrix}$$

with $I_{xx} = \int_V (y^2 + z^2)\, dm$ - infinitesimal sum of distances from the x-axis - and alike for others. But we don't need all that in 2D! Our α_i are not full vectors. If we define the x and y axes on our 2D grid, then the z axis is the one coming out of our screen. That's where the rotation lies, so we know $\alpha_i = (0 \quad 0 \quad \alpha_i)$ and thus the term inside the sum of eq. (5.1) becomes

$$(\alpha_i - \alpha_{i0})^2 I_{zz}.$$

In practice, when coding some object, we can calculate

$$I_{zz} = \iint_V (x^2 + y^2)\rho(x, y)\, dV$$

where the density $\rho(x, y)$ is a constant for homogeneous objects. It basically measures distance from the rotational axis weighted by mass.

To incorporate this, we only need to extend the mass matrix M to include I_{zz}, so

$$M = \begin{pmatrix} m_1 & 0 & 0 & 0 & \cdots \\ 0 & m_1 & 0 & 0 & \cdots \\ 0 & 0 & I_1 & 0 & \cdots \\ 0 & 0 & 0 & m_2 & \cdots \\ \vdots & \vdots & \vdots & \vdots & \ddots \end{pmatrix}.$$

This combined with the state vector update means we don't even have to change anything about our methods. We write Z in the same way and thus everything else follows accordingly.

5.3 Points in bodies

When will these rotations actually come into play? Well we obviously need them for rendering the objects at their correct angles, but they also play a big role in our constraints. Those involve velocities, and as soon as we're not talking about the velocity of the mass center - where the rotational axis is - then the rotational velocity adds some component of speed, which might then cause a change in the angle.

More specifically, if we have some point i on an object and we wish to know its velocity, we need to sum the mass center velocity and the rotational contribution, via

$$\vec{v}_i = \vec{v}_{MC} + \vec{\omega} \times \vec{r}_i.$$

So here we need the cross product, between $\vec{r}_i$ - the vector from the origin to the point - and the angular velocity $\vec{\omega}$ - the derivative of θ stored in the velocity vector. This 'origin' should for any rotation always be somewhere along the rotational axis. So unsurprisingly in this sense the origin is just the object's mass center. And since $\vec{\omega} = (0 \quad 0 \quad \omega)$, we can simplify to

$$\vec{v}_i = \vec{v}_{MC} + (-y_i\omega \quad x_i\omega) \tag{5.2}$$

in our two dimensions, with x_i and y_i the components of $\vec{r}_i$, the vector pointing from the object's mass center to this point in its body.

5.4 Changes to holonomic constraints

With this in mind, our simple holonomic constraints can become a little more complicated. If we just take two circular objects and want the distances between their mass centers to be constant, then only translation is affected. However if you want to - more realistically - connect some points on the edges, you'll have to take rotation into account. Remember the positional constraint as

$$(\vec{r}_i - \vec{r}_j)^2 - L^2 = 0.$$

Deriving to find a velocity-based constraint requires the derivatives of both $\vec{r}_i$ and $\vec{r}_j$. In general, these are no longer just $\vec{v}_i$ and $\vec{v}_j$, but instead we'd get

$$(\vec{r}_i - \vec{r}_j) \cdot (\vec{v}_i + \vec{\omega}_i \times \vec{q}_i - \vec{v}_j - \vec{\omega}_j \times \vec{q}_j) = 0.$$

Do mind that I haven't expanded the cross products down to their simplest form like in eq. (5.2). We'll expand our equations when we actually need to implement them in chapter 8. What's important is that this expression, once you've extracted the terms for each factor $(v_{ix}, v_{iy}, v_{jx}, v_{jy}, \omega_i, \omega_j)$, will yield the constraint row in A, which is used in our iterative GS cycle to update v, which will now also tweak rotational velocity.

5.5 Changes to contact and friction constraints

Our contact and friction constraints also need to be updated. That's because so far I hadn't been very specific about which velocities to use there. Both constraints try to reduce down the relative velocity, projected perpendicularly or tangentially, with restrictions on J_k. But these velocities should always be for contact points!

While the dynamics solver (constraints, time updates, GS) is one component of our implementation, so is a completely seperate collision detection system. We'll cover this system in detail in the next chapter. What's important here is that this system will result in a big list of every colliding pair of objects, as well as the contact point(s) for these collisions. We then demand that for each contact point - which should be a point that exists on both objects - the relative velocity projection is zero, with the same restrictions. It involves calculating $\vec{q}$, the vector pointing from each object's mass center to the contact point, and then saying

$$(\vec{v}_i + \vec{\omega}_i \times \vec{q}_i - \vec{v}_j - \vec{\omega}_j \times \vec{q}_j) \cdot \vec{n} = 0$$

for the contact constraint. The friction case just involves a different projection. Everything else is the same. As shown in fig. 5.1, bigger contact surfaces require more contact points. This is to make sure the movement actually looks realistic. If we didn't add the points on the right and left, the upper box might clip into the lower box by rotating around the middle contact.

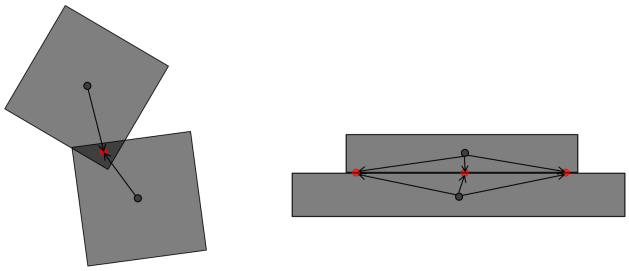

Figure 5.1: Two examples of possible collisions, the contact points (red), mass centers (gray) and relative position vectors

5.6 Example: distance constraint

To illustrate the vector mathematics behind constraints, we will derive the full update for one example constraint. The distance constraint enforces that $C = \left|\vec{d}\right| - L = 0$ where $\vec{d} = \vec{p}_2 - \vec{p}_1$ is the vector connecting a point on one body to a point on another body, thus enforcing them to always be a distance L apart. By taking the derivative of this relation, we can find the velocity-based constraint:

$$\dot{C} = \frac{d}{dt}\left|\vec{d}\right| = \frac{\vec{d}}{\left|\vec{d}\right|} \cdot \dot{\vec{d}} = \hat{d} \cdot (\dot{\vec{p}}_2 - \dot{\vec{p}}_1) = 0$$

$\hat{d}$ is the unit vector from $\vec{d}$. Since the velocities of these two points can include both the body's translational velocity and the specific rotational velocity there, we write

$$\dot{C} = \hat{d} \cdot (\vec{v}_2 + \omega_2 \times \vec{r}_2 - \vec{v}_1 - \omega_1 \times \vec{r}_1)$$

where both ω lie purely in the z-direction and $\vec{r}_i$ are the vectors from each body's mass center to the point being held at fixed distance. If we wish to extract the Jacobian row A_i, we need to expand. First notice that $\hat{d} \cdot (\omega_2 \times \vec{r}_2) = \omega_2 \cdot (\vec{r}_2 \times \hat{d})$. Since ω lies in the z-direction and the cross product on the right (between two 2D vectors) will also lie in that direction, the dot product will simply equal the product of both vectors' magnitudes, so $\omega_2 \left|\vec{r}_2 \times \hat{d}\right|$. From this we can find

$$\dot{C} = -\hat{d}_x v_{1x} - \hat{d}_y v_{1y} - \left|\vec{r}_1 \times \hat{d}\right| \omega_1 + \hat{d}_x v_{2x} + \hat{d}_y v_{2y} + \left|\vec{r}_2 \times \hat{d}\right| \omega_2$$

and read off all the Jacobian row entries. Now we expand the update for ΔJ_i via the equation

$$\Delta J_i = -\frac{A_i v^{n+1}}{A_i M^{-1} A_i^T}.$$

The numerator is just the constraint itself, so $\hat{d} \cdot (\vec{v}_2 + \omega_2 \times \vec{r}_2 - \vec{v}_1 - \omega_1 \times \vec{r}_1)$. The denominator comes out to the square of each entry in the Jacobian row multiplied by their corresponding mass inverse elements, so $\frac{\hat{d}_x^2}{m_1} + \frac{\hat{d}_y^2}{m_1} + \frac{\hat{d}_x^2}{m_2} + \frac{\hat{d}_y^2}{m_2} + \frac{\left|\vec{r}_1 \times \hat{d}\right|^2}{I_1} + \frac{\left|\vec{r}_2 \times \hat{d}\right|^2}{I_2}$. Combining and simplifying with the fact that $\hat{d}$ is a unit vector yields

$$\Delta J_i = -\frac{\hat{d} \cdot (\vec{v}_2 + \omega_2 \times \vec{r}_2 - \vec{v}_1 - \omega_1 \times \vec{r}_1)}{\frac{1}{m_1} + \frac{1}{m_2} + \frac{\left|\vec{r}_1 \times \hat{d}\right|^2}{I_1} + \frac{\left|\vec{r}_2 \times \hat{d}\right|^2}{I_2}}.$$

As for the velocity update, $\Delta v^{n+1} = M^{-1} A_i^T \Delta J_i$, we also just have to fill in the known Jacobian row entries, being careful not to forget minus signs.

- For $\vec{v}_1$ we get $\Delta \vec{v}_1 = -\hat{d} \cdot \frac{\Delta J_i}{m_1}$
- For $\vec{v}_2$ we get $\Delta \vec{v}_2 = \hat{d} \cdot \frac{\Delta J_i}{m_2}$
- For ω_1 we get $\Delta \omega_1 = -\left|\vec{r}_1 \times \hat{d}\right| \cdot \frac{\Delta J_i}{I_1}$
- For ω_2 we get $\Delta \omega_2 = \left|\vec{r}_2 \times \hat{d}\right| \cdot \frac{\Delta J_i}{I_2}$

Chapter 6

Collision algorithms

Each time-step, before any dynamics solving is performed, we require a list of every active collision. In other words, a collision system that returns a list of body pairs where those pairs of bodies are overlapping. Per pair we also want the normal (and from that the tangential) vectors - for projection in constraints - as well as the penetration depth - for Baumgarte stabilization.

This system is comprised of two main parts:

- Broad-phase detection: performs a first, general inspection to rule out pairs of bodies that are obviously not colliding. Its eventual output will still contain false positives, but the goal is to bring down the number of pairs that need to be checked more thoroughly in the narrow phase.
- Narrow-phase detection: examines every potential colliding body pair - obtained from the broad phase - to determine with 100% accuracy whether a collision is taking place. This will therefore be a lot more computationally expensive, so the more the broad phase prunes, the better. A method used here might also allow us to efficiently obtain the normal/tangential vectors and the penetration depth. If not, this is easily calculated afterwards.

Efficient collision detection is an integral part to a fast physics engine, so it can be regarded as a research branch (in mathematics or topology) just by itself. We will try to do the mathematicians justice by looking into some of the most useful techniques.

6.1 Broad-phase algorithms

6.1.1 Sort, sweep, and prune

The sort, sweep, and prune algorithm uses a sorted list of the boundaries of bodies to detect possible collisions.

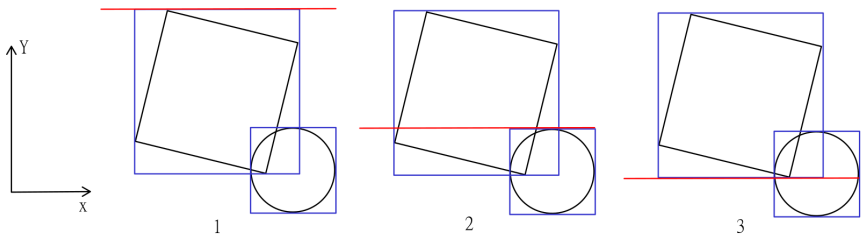

Figure 6.1: Three frames of the sweeping process between a square and a circle

The first step is to take all of your bodies and draw bounding boxes around them. These are rectangles, aligned with the given x- and y-axes, defined to be the smallest possible rectangles that completely encompass the specified body. These are shown in blue in fig. 6.1. The second step is to let a scanning line (here: red) run across the entire space for both directions. Whenever this line encounters the starting edge of a bounding box, the corresponding body is added to a tracking list. If the line encounters the ending edge, the body is removed. If at any point there are two or more bodies in this tracking list, they can be classified as potential colliding pairs. At **point 1** on the figure above, the square is added to the tracking list. At **point 2** the circle is added, while the square is still in there, which causes a collision detection. At **point 3** the square is removed from the tracking list.

Bounding boxes

To use the sweeping method, we need every object in our scene to be simplified to a simple rectangular shape. These are called bounding boxes. Many types exist, like minimum bounding rectangles (MBR) that try to find the smallest possible rectangle that contains the full shape. We are not doing that here! We're also looking for rectangles just small large enough to fit our shapes, but their edges should be aligned with our x- and y-axes. This is called an axis-aligned bounding box (AABB) and we will be using it for our simulations.

For circles, determining the AABB is simple: just draw a square around the circle. For example, the leftmost edge would have x-coordinate $x_{min} = x_c - r$. For polygons the task is a bit harder, but it comes down to taking the list of every vertex position and selecting the minimum/maximum x/y values out of all of them.

Sorting by edges

For efficiency purposes, having a real physical line scan across the scene is not the right way to go. Instead, we take the bounding boxes we found previously and sort the edges in one specific direction. When sweeping in the x direction, we take all vertical edges - each object has exactly two - and sort them from left to right, or negative to positive x. Then we just need to iterate over this list and add/remove objects to/from the tracking list accordingly. Because objects barely move across frames, we can use insertion sort as our algorithm. This is known to work well and fast on nearly-sorted lists.[18]

Sweeping

As mentioned before, if at any point the tracking list contains multiple objects, they might be overlapping. For a true overlap, this 'being in the list at the same time' will hold for both directions. So we only mark two objects as collision candidates if they appear together in the tracking list for both the x and y directions.

6.1.2 BSP, quadtrees, octrees

While its name sounds horrendous, binary space partitioning (BSP) is actually a very simple method proposed to prune down the amount of collision candidates. You recurse down a tree, starting from your entire scene of objects. At each node, the current space is cut in two. Think of it like this: you're playing hide-and-seek with your friend in a huge mansion. Since he wants to give you a fighting chance to find him, he allows you to continuously ask binary questions - only two possible answers - such as 'Are you upstairs or downstairs?' and 'Are you in the east or west wing?'. With each question you cut off approximately half of the remaining searching area, until it's become so small you can search on your own. This is the fundamental principle behind BSP. Every node narrows down our scene, until the remaining area covers only a handful of objects. Obviously, any object can only be colliding with an object in the same area partition. It thus becomes a trade-off between recursing deeper to prune more and recursing less to save on computations.

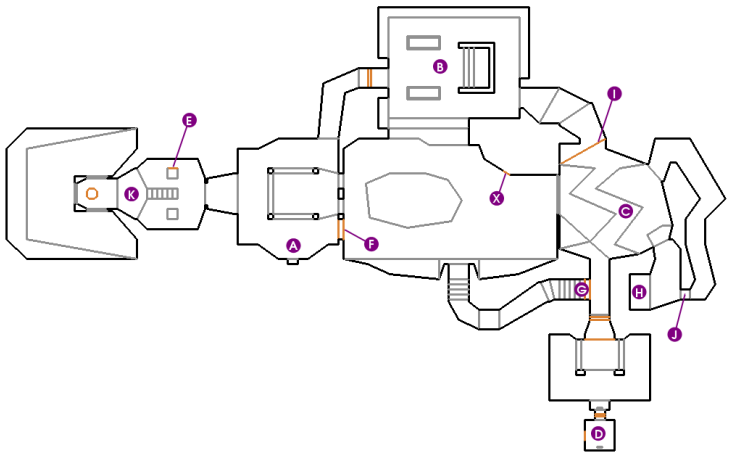

Figure 6.2: First level of Doom viewed top-down

This method is good in theory, but extremely difficult to implement in practice, mainly for the above-mentioned trade-off reason. It is more often used in pre-computed environments like the original Doom's levels (see fig. 6.2), to know which areas to compute and which to set inactive.[13]

Quadtrees and octrees are extensions of the binary space partitioning method. Instead of splitting objects into two groups at each layer, we split into four or eight at each layer. This is once again done until a small amount of objects remain per cell, where we can assume those bodies form potential collision pairs.

6.2 Narrow-phase algorithms

After a set of collision candidate pairs has been generated in the broad-phase search, it gets passed on to any of our narrow-phase algorithms. They will determine, with required 100% accuracy, which of these are actually overlapping. This finalized list can be used to find just how bad the overlap is and what the normal/tangential vectors are, for use in dynamic solving.

6.2.1 Separating Axis Theorem

Theorem

The separating axis theorem (SAT) is a theorem in geometry that gives us a way to determine whether two convex shapes are penetrating or not.

Theorem 6.2.1 (Separating Axis Theorem). Two convex shapes A and B do not overlap if and only if there exists a separating axis.

Proof. The proof of this theorem lies far beyond the scope of this introductory course. It is however an extremely intuitive statement. Therefore we will accept this proof by intuition. □

Let's clarify a bit. What are convex shapes and what are separating axes?

A convex shape is a shape for which there exists no straight line that crosses its edges more than twice. The opposite is called a concave shape. The difference is visualized in fig. 6.3.

(a) A convex shape (b) A concave shape

Figure 6.3: Difference between convex and concave shapes [2]

A separating axis is any axis where, when the two shapes are projected onto it, their projections do not overlap. The theorem basically states that if such a separating axis exists, the shapes do not overlap. It also works bidirectionally: if the shapes do not overlap, you are guaranteed to find an axis like that. Or you could invert it and say that if you cannot find a separating axis, the shapes must be overlapping.

Finding separating axes

At this point you might be wondering just which axes we will be checking to determine collision. It would be foolish to just pick N random axes at equidistant angles and hope for the best. In actuality, you only need to check the axes that are normals to the shapes' edges. That means that for two rectangles, you only need to try a maximum of 8 axes - a normal on each side. Since SAT is a falsification theorem, once we find one separating axis we can immediately return

not-overlapping and quit the search. It is only when we've gone through every possible axis with no luck that we can conclude they must be overlapping.

In general, if you're checking for overlap between an n-polygon and an m-polygon, you need to check $n + m$ axes for possible separation. But what if you have a circle? You could argue those are polygons with an infinite amount of sides, but we clearly cannot check infinite axes. Instead, we check the axis formed by connecting the center of the circle to the closest vertex - that is the closest corner of the other shape. If you have two circles, you evidently just need to check the axis connecting both their centers.

Mathematical details: projection

We describe any axis using the angle θ it forms with the x-axis. If $\theta = \frac{\pi}{2}$ we would thus have a vertical line. To project any point (x, y) onto this axis, we take the vector given by (x, y), take the dot product with the unit vector $(\cos\theta, \sin\theta)$ and create a vector in the unit direction with the dot product result as its magnitude. So the projection is given by

$$[x', y'] = [(x\cos\theta + y\sin\theta)\cos\theta, (x\cos\theta + y\sin\theta)\sin\theta] .$$

This equation will be used to project single points onto axes. These could be the vertices of polygons or the centers of circles. In both cases, a simple extension like combining all vertex projections into a line or extending a radius length outwards gives us the shadow of the entire shape on the axis.

Mathematical details: separating axes

We already covered which specific separating axes we needed to check for collisions. In most cases we will end up with a vector (x, y), like the one connecting two circle centers. To get our angle θ from this to define the possible separating axis, we obviously just calculate $\theta = \arctan\frac{y}{x}$, making sure to split off the case where x nears zero, to prevent division by zero.

In the case of a polygon, those normal vector axes are easily found by just getting the edge vector, calculating its θ and adding (or subtracting) $\frac{\pi}{2}$.

The case of concave shapes

The Separating Axis Theorem, as defined before, does not work for concave shapes. This is intuitive, because a concave shape with a little cutout and another shape poking into that cutout would be recognized as a collision by SAT, while it is not one. To fix this, we can divide any concave shape into convex sub-shapes (see fig. 6.4). We then check for proper SAT collision between every pair of A's sub-shapes and B's sub-shapes. If none of the sub-shapes of A overlap with any of the sub-shapes of B, there is no collision. So while a solution for concave shapes exists, we won't be implementing it in our final engine to spare the sub-shape hassle.

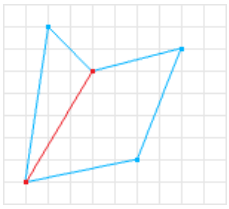

Figure 6.4: A concave shape decomposed into convex sub-shapes

6.3 Contact manifold determination

The final step our collision engine should perform is creating a contact manifold for each colliding pair of objects. The usage of the word "manifold" here will most likely upset mathematicians, since all this is is a concise object carrying all necessary data about the contact, to give to our dynamic engine. So we need to specify the two objects, the normal vector, the tangent vector, and the penetration depth. The last three are used to properly implement the contact and friction constraints, where we need the penetration depth for Baumgarte stabilization.

But that's not all: as we mentioned before, some collisions might require multiple contact points to be taken into account. Like two rectangular objects placed on top of each other: if you use only one contact point and only do constraint solving for that one, nothing is preventing the solver from rotating the rectangles around that point, into each other. So in those cases multiple contact points and equivalently multiple contact/friction constraints per collision can help.

And finally there is the exact location of the contact. This should be a point that lies on both objects. We need its location, because the relative vectors $\vec{q}_i$ that point from each object's center to this point are used for calculating the point's velocity in both bodies - specifically the rotational part $\vec{v}_{rot} = \vec{\omega} \times \vec{q}_i$.

6.3.1 Defining the normal and tangent vectors

For the normal vector $\vec{n}$, which is the direction in which the objects are penetrating each other, you might assume we just draw a vector from one body's center to the other body's center. This might work for simple objects like squares, but in most cases the centers we choose arbitrarily don't have anything to do with penetrations.

Luckily SAT comes to our aid once again. Since we know there is a collision, we can conclude that during narrow-phase, we did not find any separating axes. But we can look at which of our axes had the smallest overlap between the body projections. That axis will be the normal vector direction we're looking for. This is intuitively shown in fig. 6.5.

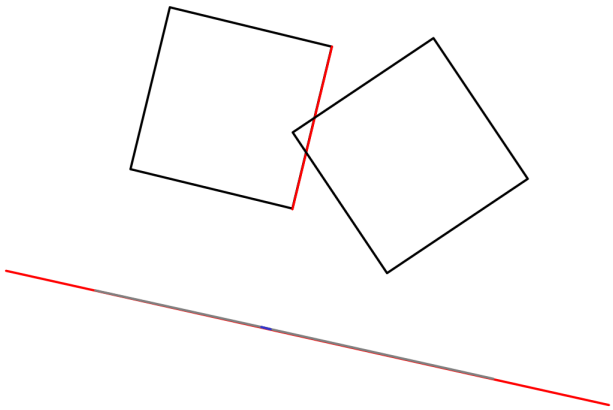

Figure 6.5: Two penetrating black squares and the red axis (normal of red square edge) that minimizes the blue overlap between the gray projections of the squares

The tangent vector $\vec{t}$ is just the perpendicular to $\vec{n}$, which is very easy to calculate.

6.3.2 Defining the contact point(s) and relative vectors

Next up is determining the contact point(s) for this collision. Remember that each contact point will result in a separate contact and friction constraint, so it can build up quickly.

We have to introduce some nomenclature first. We found $\vec{n}$ by figuring out which axis had the least projection overlap. This axis always corresponds to an edge of a polygon. In fig. 6.6 it belongs to the leftmost square. We will call this square the reference object, and the other the incident object. Furthermore, the red edge here is called the reference edge. The incident edge (blue) is the edge for which the dot product of its outward normal with the reference edge's outward normal is smallest - possibly most negative. In other words, this incident edge is the edge most opposite or anti-parallel to the reference edge.

The incident edge has two vertices, shown as green circles here. These vertices are the only possible contact points for our determination. We can clearly tell that the top vertex doesn't qualify as a contact point, but let's determine that procedurally. For each of the green vertices we check the following:

- The reference edge also has two vertices. Now create a 3D plane through each of these vertices, with the normals pointing towards the opposing vertex. In 2D you would see those yellow axes. We now eliminate any green point that is not "inside" both of these planes - the "inside" of a plane is the side to which its normal points. In our example that means eliminating any green point not between the two yellow axes.
- If a green point happens to be eliminated, like our top one, we replace its candidacy by a pink point which is just the intersection between the incident edge and the plane whose "inner-ness" we did not respect - kinda like snapping it back in. This step makes sure any contact points we determine are within the logical boundaries of the reference edge's horizontal.

- The final check is to eliminate any of the remaining points that are inside of the reference edge. The reference edge (red line) obviously also determines a 3D plane, with the normal outward the shape. A point inside of this plane - side outward normal is pointing to - is outside of the reference object's shape. That can clearly not be a contact point, those must be in both shapes! So we only keep the points that are "outside" the reference plane.
- Since we started with 2 possible points - the vertices of the incident edge - the options are that we're left with 0, 1, or 2 contact points. In fig. 6.6 there would be only one. For a collision between the edges of two polygons (like two stacked squares), there would be two. If you don't find any contact points, you must have screwed up somewhere in the earlier steps like the SAT collision determination!

This is an algorithm very similar to the Sutherland–Hodgman algorithm, used to clip a polygon with another polygon.[17]

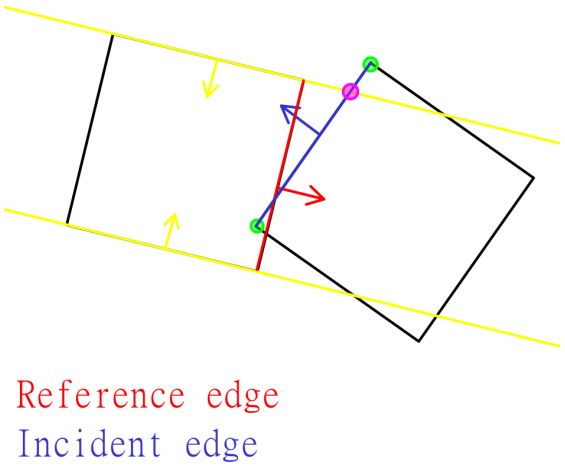

Figure 6.6: Two penetrating squares, the red reference edge, blue incident edge, yellow planes, and corresponding outward normals

Once you have your contact points, you can use them to find the relative vectors pointing from each body's center to them - for rotation in relative velocity.

6.3.3 Defining the penetration depth

Finally there's the penetration depth. Because of its use in Baumgarte stabilization for our contact constraints, each contact point has a different penetration depth to calculate. The reasoning is simple, though. The penetration depth is the perpendicular distance from the contact point to the reference edge. In fig. 6.6 that'd be the perpendicular distance between the bottom green point and the red reference edge!

6.4 Preventing tunneling

Something that comes up often when discussing collisions in simulations or games, is the effect of tunneling. Imagine you have a thin wall and an object heading straight for it at a rapid pace.

It's possible that given its high velocity, a position time-step could make it go from 'right in front of wall' to 'right behind wall' - see fig. 6.7. This phase-through is what we need to prevent.

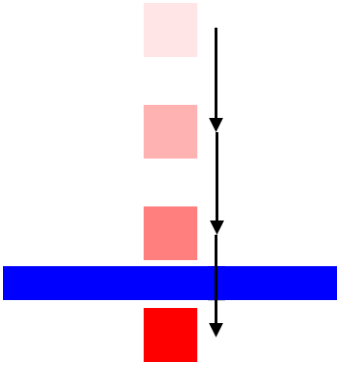

Figure 6.7: A fast square tunneling through a thin wall
[9]

The effect can be caused by a combination of high speeds, large time-steps Δt, and thin object collisions. Solving it, however, is not as simple as lowering Δt and hoping a more time-detailed simulation will cause collision preventing tunneling.

The real solution to this conundrum will be Continuous Collision Detection (CCD). As the name implies, we will use continuous approximations to zoom into time-steps and figure out which objects will cross each other. The name is also a bit of a lie though, since doing something continuous in programming is by definition impossible - that'd require infinitesimally small time-steps...

CCD is a trade-off: we're having to come up with approximation methods to detect collisions before they happen discretely, and so we're bound to make mistakes. Simulations can never be 100% correct. Therefore we'll only briefly cover the two main ways CCD can work, as well as their downsides. This will not be implemented in our final engine, as it only fixes rare circumstances and its benefits do not outweigh the effort. Some might say its implementation is left as an exercise for the reader.

6.4.1 Sweep-based CCD

The first technique is called sweep-based CCD. As the name implies, we take an object and its velocity, then sweep out its predicted path for this time-step and check if any other objects are on that path. If there are, we estimate when that collision would happen in time and split the current time-step into sub-steps accordingly, making sure the collision actually occurs and gets detected properly.[12]

This sounds amazing on paper, but has a lot of downsides:

- It requires its own (preferably) fast collision algorithm, between swept paths and other objects.
- It demands a ton of computations, especially if there are a lot of objects present in your simulation. The computational speed can also be tanked by faster objects, as they sweep

out larger areas in the same time-step.

- It is unable to prevent tunneling for rotational movements, due to the extra simulating that would require.

6.4.2 Speculative CCD

The second technique is called speculative CCD. For any two objects we can calculate their relative velocity and project it onto the normal between them. This was used before for contacts, when objects physically collided with each other - we then forced this velocity to be zero or positive. If objects are not yet penetrating, but they might cross over each other, we can introduce a speculative contact constraint. It enforces

$$\vec{n} \cdot (\vec{v}_1 - \vec{v}_2) \geqslant -\frac{\Delta r}{\Delta t}$$

meaning the relative velocity *can* be negative, but only up to the specified amount. The amount in question is just the speed you'd need to bridge the gap (Δr) between the two objects in one time-step (Δt). So if two objects are 5 units apart and a time-step is 1 second, the speculative constraint just says these objects cannot have a normal relative velocity higher than 5 units per second (in the negative direction).

But do we need to add these speculative constraints for all pairs of objects? Obviously not! If in the current frame the two objects have too little relative velocity to cross the gap in a single time-step, they probably shouldn't be considered for a speculative constraint. I say 'probably' since the velocities can change in the solver, so we need a bit of margin. Let's say that if $\vec{n} \cdot (\vec{v}_1 - \vec{v}_2) \leqslant -\frac{\Delta r}{\Delta t} + m$, with m an error margin, we should consider adding a speculative constraint.

There's also the fact that these non-colliding pairs we want speculative contacts for, might not even be selected in the broad-phase collision detection. So we also change the broad-phase step: extend an objects AABB to cover both its current position and its predicted position in the next frame, using the current velocity. This will make the new AABB overlap anything that the object might tunnel through, and broad-phase detection will pass it on to narrow-phase. Inside of narrow-phase, we still start by using SAT for the final elimination, but instead of throwing all other candidates away immediately, we check the condition from before to maybe add that speculative constraint.[11]

Chapter 7

Designing an engine

7.1 Full simulation layout

Take a breather, because at this point we've covered basically everything you need for your first genuine 2D physics engine. To recap, the entire structural diagram is visualized neatly on fig. 7.1.

During the process of explaining each component in the previous chapters, we didn't always go too in-depth on practical or computational implementations. That's what this chapter is for! Let's go over the exact remaining steps for engine implementation.

1. We decide a programming language to implement the engine in.
2. We design before we program, using class diagrams and layouts.
3. We create all necessary files, empty implementations, and specifications.
4. We implement from root to most dependent, building up the engine.
5. We write a simple renderer.
6. We perform a multitude of tests.
7. We recreate the first level of Angry Birds with the engine.

Something to note is that we obviously won't be covering every single line of code in this chapter: far from it. We'll cover some important design aspects and some technical details, but the best way to understand the full implementation is to either read the source code or give it your own shot!

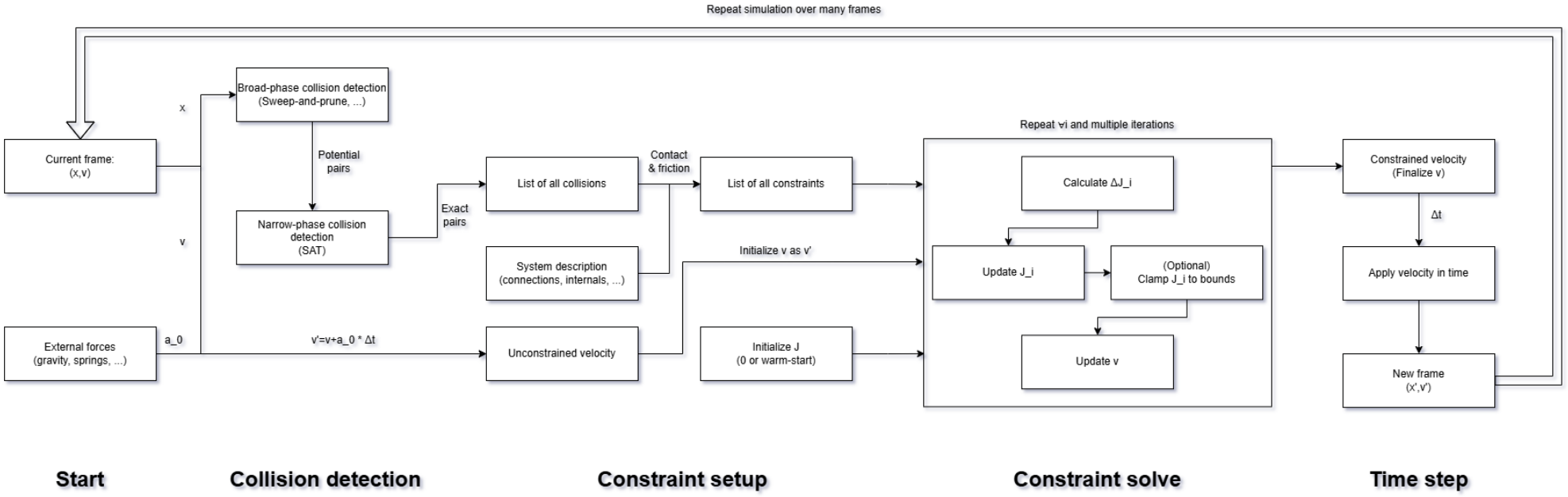

Figure 7.1: Full schematic representation of the 2D physics engine (WIP recheck with updates)

7.2 Choice of programming language

This 2D physics engine can hypothetically speaking be implemented in any and all programming languages. However, given the nature of the system, some will make this job easier than others.

Physics engines are all about objects and how they interact with each other dynamically. This is a perfect match for object-oriented programming languages (OOPs). That's why the language we choose should probably be one of those. We can then design the class diagram first, with all necessary classes, methods, associations, inheritances, etc.

That still leaves us with quite a few options, but the main ones to consider are Python, C, C++, and Java. Python would be a beginner-friendly choice, but we won't be using it because its object-oriented programming tools are more like an imitation of OOP and not a real version of it. Python also doesn't benefit from compiler speedups or proper type-checking to prevent fatal errors. Since I enjoy the outside world, we also won't be using good old C. So the decision is between C++ and Java.

A popular poll would probably give C++ the lead, since it is better suited for mathematical and scientific programming. We however will be choosing Java, partly because it's more user-friendly and mainly because I followed a Java course in my undergraduate degree. Please note that this decision has no tangible effect on our understanding of the underlying principles, both the physics and the implementation. Porting the engine from Java to C++ would be light work.

7.3 Class diagram design

Before we write a single line of code, we should think about what classes we'll be using, how they are related and how they (optionally) inherit from one another. We'll begin with the base classes that do no inheriting - apart from default inheritance of *Object*. After that we'll cover the classes that do inherit, which are mainly specializations or implementations of those base classes. For (mostly) mathematical and bookkeeping purposes, we'll also need a few helper classes.

7.3.1 Base classes

- **World**: The *World* class is one of two main carriers of the physics simulation. Anything physics-related that should happen is controlled here. That means adding bodies, adding forces, adding constraints, etc.
- **PhysicsLoop**: This class is the other main simulation handler, but it takes care of the technical side of things. This includes the CPU thread that runs the simulation at a high frame-rate, the rendering pipeline (and program window), and user control over starting and stopping the simulation. Each *PhysicsLoop* instance is unidirectionally associated to a *World* object, such that it can force the physics updates and get information for rendering.
- **Body**: This class is the main definition of what a body in our physical world represents. It is an abstract class, meaning you can only create instances of inheriting classes (such as *Polygon*) that implement detailed methods. It specifies a ton of important rules, such as how positions are stored - especially relative to where the user thinks an object is - and

what other data needs to be kept. These include mass, velocity, angle, angular velocity, and moment of inertia.

- **LocalForce**: This class is the representation of one local force, which means it applies to a limited subset of the World's full body list. This is reserved for things like the spring force. On this abstract level, it doesn't do much but define the required methods to for example apply acceleration.
- **UniversalForce**: This class is very similar to *LocalForce*, but it is intended for a force that applies to every body in the simulation. That's why it keeps track of all those bodies in its own list, that must be updated properly each time you add or remove a body to or from the associated *World*. It also once again defines an abstract method to apply its acceleration, defined by inheritors.
- **Constraint**: This is the main constraint class. It specifies abstract methods for calculating ΔJ, initiating constraints at each time-step, and also performing one full update of the constraint (including Δv application).
- **BroadPhaseCollisionSystem**: This abstract class specifies what a broad-phase collision system should be capable of. Namely it keeps track of all the simulation's bodies on its own, which needs to be updated properly each time you add or remove a body for the underlying *World*. It then also specifies a method that returns a list of all possibly colliding pairs of bodies, which will later be passed on to a narrow-phase collision system.
- **NarrowPhaseCollisionSystem**: Same as the broad-phase collision system, but for the final checks. It mainly defines a method that takes a potential colliding pair (or list of them) and returns all associated contact manifolds, basically confirming or denying whether a collision is actually occurring.

7.3.2 Inheriting classes

- **StaticConstraint**: Inherits from *Constraint*. This is still abstract, but now specifically designed for constraints that are user-defined and last multiple steps of the simulation. An example would be the distance constraint, which is added or removed by the user.
- **DynamicConstraint**: Inherits from *Constraint*. Unlike *StaticConstraint*, this one is aimed at constraints that are dynamically created each time-step, like *ContactConstraint* or *Friction-Constraint* being added when a collision happens that time-step.
- **SweepAndPrune**: Inherits from *BroadPhaseCollisionSystem*. This is the implementation of the main broad-phase system we covered, the sweep-and-prune algorithm that sorts edges and roughly detects possible collisions using AABB overlapping.
- **SAT**: Inherits from *NarrowPhaseCollisionSystem*. This is the implementation of the main narrow-phase system we covered, the one using the Seperating Axis Theorem. It will be one of the most mathematically complex classes to work out.
- **Circle**: Inherits from *Body*. The class for creating circular bodies, nothing too special.
- **Polygon**: Inherits from *Body*. The class for creating polygonal bodies, which should cover anything that isn't a circle or a shape only mathematicians use. Due to the high degree of general-ness required to write methods that work for any polygon, this will also be a tough class to implement.

- **Rectangle**: Inherits from *Polygon*. A special (but useful) polygonal case where the four points of the rectangle - calculated from the user's request - are just passed through to *Polygon*.
- **Square**: Inherits from *Rectangle*. This one speaks for itself.
- **DistanceConstraint**: Inherits from *StaticConstraint*. This one speaks for itself.
- **ContactConstraint**: Inherits from *DynamicConstraint*. It handles the collisions themselves and prevents things from acting like ghosts.
- **FrictionConstraint**: Inherits from *DynamicConstraint*. It handles friction associated to a collision and thus needs to keep track of which *ContactConstraint* it is linked to - this determines the maximum friction.
- **AirResistance**: Inherits from *UniversalForce*. This one speaks for itself.
- **Gravity**: Inherits from *UniversalForce*. This one speaks for itself.
- **Spring**: Inherits from *LocalForce*. This one speaks for itself.

7.3.3 Helper classes

These classes stand on their own and serve the purpose of carrying information and making our calculating lives easier. We won't cover every single one and we won't go more in-depth than what is mentioned here.

- **Vector2**: A lot of vectors and vector operations are necessary to power an efficient physics engine. This class is a custom implementation of a 2D vector, including sums, dot products, cross products, etc.
- **ContactManifold**: Once a narrow-phase collision system verifies a collision, each contact point (up to 2) is specified as a *ContactManifold*. Each of these will create and control one contact and one friction constraint, so they carry information about the contact point location, relative vectors to this point, the penetration depth, normal vector, and tangent vector. All that information must therefore also be discovered by the narrow-phase collision system!
- **PotentialCollidingPair**: This is a very basic data class that carries knowledge from the broad-phase to the narrow-phase of collision detection.

7.4 Implementing classes

Now that we know which classes (and therefore which file structure) to use, let's select some of the most important technical details used in the implementation, together with some code fragments.

7.4.1 Bodies

For bodies, the first problem we encounter is choosing a coordinate system from within. For the sake of physics being more simple, we made sure the positions stored by the object always refer to where the mass center of the body would be. The issue is that, in most cases, the user expects something different. When they define a circle, they're probably specifying the center, which would equal the mass center and thus cause no problems. But when they define a rectangle or any polygon, they have no way of knowing where the mass center will be. Hence why we differentiate the user origin and the real origin. The real origin is the mass center, which corresponds to all the stored numbers. The user origin is where the user wants their object to be placed. For a rectangle this is the top-left corner: if the user specifies a position of (0,0), we must calculate where the mass center will be and set the stored position to that instead. We also need to keep track of some vector pointing from the real origin to the user origin, such that a position update specified by the user can be handled quickly and does not require re-calculating the mass center each frame.

For circles, there are no other big problems to cover. But polygons are a whole different story. First of all, remember that the SAT algorithm only works for convex polygons. So when the user specifies a polygon by giving an ordered list of vertex coordinates, we must first check this requirement. Furthermore the direction in which the user defines the vertices (often called the winding direction) can determine which way the normals of the edges face. To make our lives easier and our code more general, we choose one winding direction and reverse the list if the user specified them differently.

In a lot of later applications - mainly the collision system - we will need access to the live coordinates of each vertex in our polygon. That's why, when the polygon is first created, we store unrotated vectors from the real origin (mass center) to each vertex. At any point in the simulation we can just take the polygon's position, rotate those stored vectors and add them to find any vertex coordinate.

Calculating the mass center, area, and moment of inertia for any given polygon is not pleasant. The equations exist, because of course they do, but they're quite ugly and long. A simple Google search or a look at the source code gives you the equation, but let's cover the moment of inertia as an example:

- We begin by calculating the polar moment of inertia (mass-independent) around the system origin (0,0):

$$J_O = \frac{1}{12} \sum_{i=0}^{N-1} (x_i y_{i+1} - x_{i+1} y_i) \left(x_i^2 + x_i x_{i+1} + x_{i+1}^2 + y_i^2 + y_i y_{i+1} + y_{i+1}^2\right)$$

- Then we shift it to the real origin (mass center) using the parallel axis theorem:

$$J_{com} = J_O - A \left(c_x^2 + c_y^2\right)$$

- And finally convert it to the actual moment of inertia:

$$I = J_{com} \cdot \frac{m}{A}$$

7.4.2 Constraints

All constraints follow a fixed design pattern. They all have a J value that gets reset after each frame - no warm-starting was implemented. Then there is one main method that handles a single update of the constraint:

```
public void updateConstraint() {
        double J_old = J;
        double deltaJ = calculateDeltaJ();
        J += deltaJ;
        capJ();
        double realDeltaJ = J - J_old;
        updateVelocity(realDeltaJ);
}
```

As you can see, the possibility of clamping J is incorporated right into the update cycle - velocity updates use the actual change. This is mainly used by the contact and friction constraints: static constraints like the distance constraint will never need this and so *capJ()* is set to do nothing in that set of constraints.

All that the inheriting classes have left to do is specify implementations for *calculateDeltaJ()* and *updateVelocity(realDeltaJ)*. We have already covered the exact equations for the distance constraint in 5.6. The equations for the contact and friction constraints can be derived in an analogous way, but there are some details for the contact constraint.

The update for the contact constraint ends up being

$$\Delta J_i = -\frac{\vec{n} \cdot (\vec{v}_2 - \vec{v}_1)}{\frac{1}{m_1} + \frac{1}{m_2} + \frac{|\vec{r}_1 \times \vec{n}|^2}{I_1} + \frac{|\vec{r}_2 \times \vec{n}|^2}{I_2}}$$

with the numerator representing the relative velocity along the normal direction, which is what we're forcing to be zero. But there's an important detail here we haven't covered: bounciness. With the current constraint, you would be fine with two colliding objects coming to a complete stop against each other. But what if we want full bouncing, meaning they smash into each other and fly apart again? This requires us to not demand $\vec{n} \cdot (\vec{v}_2 - \vec{v}_1) = 0$, but rather equal to a multiple of the flipped relative velocity. To be precise: at the start of a frame, when initiating the contact constraint, we take a look at the current normal relative velocity, and with some restitution or bounciness $\epsilon \in [0, 1]$ we say the bounce velocity $v_b = -\epsilon \vec{n} \cdot (\vec{v}_2 - \vec{v}_1)$. By then enforcing the constraint to be $\vec{n} \cdot (\vec{v}_2 - \vec{v}_1) - v_b = 0$ - where the left part updates through iterations but the bounce velocity does not - we can end up with some realistic escape-bounce velocity. This yields

$$\Delta J_i = -\frac{\vec{n} \cdot (\vec{v}_2 - \vec{v}_1) - v_b}{\frac{1}{m_1} + \frac{1}{m_2} + \frac{|\vec{r}_1 \times \vec{n}|^2}{I_1} + \frac{|\vec{r}_2 \times \vec{n}|^2}{I_2}}$$

Furthermore, we will only do this if the speed at which they're colliding into each other is large enough - or in this case negative enough projected onto the normal - because for small and slow collisions this fix is unnecessary and unstable. Also note that none of this matters for the equivalent pseudo-velocity updates in NGS, since there is no starting velocity there - so no goal bouncing velocity - and it only tries to stop them from penetrating position-wise there.

For the friction constraint, there is no Baumgarte bias. We also don't do any fancy bouncing logic, so we simply have to derive that

$$\Delta J_i = -\frac{\vec{t} \cdot (\vec{v}_2 - \vec{v}_1)}{\frac{1}{m_1} + \frac{1}{m_2} + \frac{|\vec{r}_1 \times \vec{t}|^2}{I_1} + \frac{|\vec{r}_2 \times \vec{t}|^2}{I_2}}$$

and apply everything like before. In this case, J is being capped to a friction maximum determined by the associated contact constraint. That's handled by this piece of code:

```
public void capJ() {
        double limit = frictionCoefficient * Math.abs(myContactConstraint.getJ());
        if (J > limit) {
                J = limit;
        } else if (J < -limit) {
                J = -limit;
        }
}
```

Since friction has no bias - nothing related to position to fix, the pseudo-velocities starting at zero will never contribute a meaningful change. Their methods are just left empty here.

7.4.3 World

Besides storing forces, constraints, and bodies, the *World* class is also responsible for the main solver loop.

```
public void step(double dt, PhysicsLoop linkedLoop) {
        [...]
}
```

Let's go over the steps it takes, one by one. First it performs both broad- and narrow-phase collision detection.

```
// Perform broad phase collision detection
List<PotentialCollidingPair> potentialPairs = broadPhase.getPotentialCollidingPairs();

// Perform narrow phase collision detection
List<ContactManifold> contactManifolds = narrowPhase.getContactManifolds(potentialPairs);
```

As you can see, the information is passed on directly between the two. With this list of collisions and an existing list of static constraints that persist through the simulation, we can make a list of every single constraint for this frame of the simulation.

```
// Initialize list of all constraints
List<Constraint> constraints = new ArrayList<>(List.copyOf(staticConstraints));
List<ContactConstraint> contactConstraints = ContactConstraint.createConstraints(contactManifolds);
for (ContactConstraint c : contactConstraints) {
        constraints.add(c);
        FrictionConstraint associatedFrictionConstraint = c.createFrictionConstraint();
        c.setMyFrictionConstraint(associatedFrictionConstraint);
        constraints.add(associatedFrictionConstraint);
}
```

The included for-loop ensures that the relevant contact and friction constraints are indeed linked. After that they're dumped into one big list of every constraint, ready for solving. Before we solve constraints however, we need to deal with the unconstrained acceleration - all this is still based on Gauss's principle!

```
// Calculate unconstrained velocity
for (UniversalForce universalForce : universalForces) {
        universalForce.applyAcceleration(dt);
}

for (LocalForce localForce : localForces) {
        localForce.applyAcceleration(dt);
}
```

After that the constraints can all be initiated, meaning their biases are calculated and they're reset to zero.

```
// Initialize every constraint's solving tech
for (Constraint c : constraints) {
        c.initConstraint(beta, dt);
}
```

Now we perform the velocity-based constraint updates, referred to here as the Projected Gauss-Seidel updates. We loop over every constraint a set amount of times, determined via the hyperparameter *detail*.

```
// PGS Velocity Iterations
for (int i = 0; i < detail; i++) {
        for (Constraint c : constraints) {
                c.updateConstraint();
        }
}
```

After this it's time for the Non-Linear Gauss-Seidel updates that fix positional constraints. These special velocities are started at zero like discussed, and then a loop analogous to before is run.

```
// Reset pseudo-velocities before NGS loop
for (Body b : bodies) {
        b.setPseudoVelocity(new Vector2());
        b.setPseudoAngularVelocity(0);
}

// Accumulate NGS pseudo-velocity steps over iterations
for (int i = 0; i < detail; i++) {
        for (Constraint c : constraints) {
                c.updatePseudoConstraint();
        }
}
```

Then we apply the pseudo-velocities...

```
// Apply the accumulated pseudo-velocities to fix positions cleanly
for (Body b : bodies) {
        if (b.isFrozen()) {
                b.setPseudoVelocity(new Vector2());
                b.setPseudoAngularVelocity(0);
                continue;
        }
        b.setPosition(b.getPosition().plus(b.getPseudoVelocity()));
        b.setAngle(b.getAngle() + b.getPseudoAngularVelocity());
}
```

... and then the regular velocities - this order does not matter.

```
// Perform standard time step integration
for (Body b : bodies) {
        if (b.isFrozen()) {
                b.setVelocity(new Vector2());
                b.setAngularVelocity(0);
                continue;
        }
        b.setPosition(b.getPosition().plus(b.getVelocity().times(dt)));
        b.setAngle(b.getAngle() + dt * b.getAngularVelocity());
}
```

Now that a time-step is completed, we can tell each body to update its internals to prepare for the next one. This means updating the AABB, as well as more complex stuff like polygons re-calculating where their vertices are in the world. We also do this for the broad-phase collision system, because things have moved and it may need to update internals - like the sweep and prune algorithm having to re-sort the body edges using insertion sort.

```
// Update each body internally for next step
for (Body b : bodies) {
```

```
        b.updateInternally();
        b.updateAABB();
}

// Update the broad phase system for next step
broadPhase.update();
```

7.4.4 Physics loop

As mentioned before, the *PhysicsLoop* class handles not the physics, but the running and rendering of the program for the user. This gets very technical, especially when it comes to rendering. We will cover the main loop and assume we can use Java threads to run this at a good pace, as well as assuming we can obtain a method to draw a pixel on the screen.

```
public void run() {
        lastTime = System.nanoTime();

        while (running) {
                long now = System.nanoTime();
                double elapsed = (now - lastTime) / 1_000_000_000.0;
                lastTime = now;

                if (elapsed > 0.25) elapsed = 0.25; // Lag spike protection
                accumulator += elapsed;

                // Step the physics
                while (accumulator >= targetDt) {
                        world.step(targetDt, this);
                        accumulator -= targetDt;
                }

                // Blit everything to screen
                render();

                // Tiny sleep to prevent running hot at 100% CPU thread starvation
                try {
                        Thread.sleep(1);
                } catch (InterruptedException e) {
                        e.printStackTrace();
                }
        }
}
```

Most of this is straight-forward:

- The code determines how much time has elapsed since the last run of the thread
- It adds this elapsed time to an accumulator (with some lag spike protection)
- Instead of telling the world to run a step with the elapsed time, we run steps at the exact Δt we want, meaning we strip those off the accumulator until its value is below Δt - this implies multiple world steps may happen during one run of the thread
- Once one or more world steps have been taken, we render the whole simulation on the screen
- To prevent the thread from breaking, we enforce a delay of at least one nanosecond

7.5 The hyperparameter problem

Throughout the development of the engine so far, we have introduced quite a few hyperparameters. These are parameters meant to control how strict or lenient a feature is on both the

accuracy and numerical stability of the simulation. The existence of these abstract knobs is a pity, since it emphasizes the fact that rigid-body simulations will remain numerical and never be 100% physically conserving of energy or momentum, but so be it. Let's cover all of them, why they exist, and which values were chosen for them.

- Δt: this is the time-step we take for world updates. It speaks for itself and has a pretty linear effect on the quality of the simulation. Make it too big and you'll need to run less updates but you'll lose accuracy when objects teleport too much each frame. Make it too small and you'll need to run more updates but you'll get a smoother simulation. For our basic simulations we used a target FPS of 200, meaning $\Delta t = 0.005$s.
- β: this is the Baumgarte parameter. When enforcing a positional bias, this factor determines how hard to apply a correction force. Setting it too high means that when objects collide, the correction can overcompensate and send things flying. Setting it too low might mean the correction becomes weak, allowing the constraint to be ignored. A stable and widely accepted value, which we used, is $\beta = 0.3$.
- N: better known as *detail* in the code, this is the parameter that defines how many iterations to run for an update of each constraint. This goes for both PGS and NGS, both perform N iterations over all constraints. Adding warm-starting for our constraints would soften the need for high detail - since we don't have to restart from zero - but in our case we went with a value of 5. From testing it seems like even a value as low as 1 yields acceptable simulations with barely any errors, but this is most likely because Δt being very small compensates this hyperparameter.
- v_b: connected to the bouncing velocity is a boundary. As mentioned, we only enforce an outward bouncing velocity with restitution if the collision velocity (relative and projected onto the normal) is large enough. In our case, "large enough" means bigger than 0.5.

7.6 Pixel rendering

A second-to-last piece of the puzzle is how we will render any shape onto the screen. Transforming internal coordinates to on-screen coordinates (where the origin is top-left and the y-axis points down) is not the hard part, but determining which pixels should be on and off for a given shape is.

7.6.1 Filling polygons using scanline rasterization

For polygons, we will use scanline rasterization. The algorithm I'm about to describe only works for convex polygons, and although the extension to concave ones is minimal, it doesn't matter since we only support convex polygons due to SAT's limitations.

- Get a list of every vertex (screen) coordinate
- Sort the vertices from highest to lowest (y)
- Take the highest vertex and add the two edges it connects to the Active Edge List (AEL)
- Set the entry and exit values to that highest vertex's x coordinate
- Start scanning down

  - Use the edges in the AEL and their directions to find their positions for this y level
  - Use those to find the new entry and exit values
  - On this y level, color in every pixel between the entry and exit points
  - If at any point your scanning y crosses the end of an edge, update the AEL with the following edge
- Continue until the final two edges meet and stop drawing

This algorithm works wonders for drawing any polygon to the screen. The details left out here are (a) what should happen if the highest vertex is off-screen and (b) how to handle perfect horizontals. In each case, running simple tests and adding secure limits for the drawing will do the trick. The final *drawPolygon()* method can be found in the *PhysicsLoop* class for those reading the source code.

7.6.2 Sub-pixel antialiasing

Pixel density is finite and a screen is mathematically unable to display diagonal lines without showing some aliasing - meaning your edges will look jagged and rough. To smoothen the look, we can trick our eyes by estimating how much of an edge pixel is actually covered by the shape and coloring it an according shade between black and white. To keep it simple, we used sub-pixel antialiasing. Each pixel is split into four horizontal subpixels (thin rectangles) where we can use basic maths to determine how many of those four subpixels are fully covered by the shape, according to the entry and exit values from before. The resulting coverage $\frac{n}{4}$ allows us to assign a color between black (not covered, background color) and white (fully covered, shape color).

7.6.3 Rendering circles

For circles the rendering part is a lot simpler. We start by drawing a square around the circle's perimeter. This allows us to easily limit how many pixels we should iterate over. Then each pixel checks its center's distance to the circle center. If this distance $d < r - 0.5$ it should be fully covered. If however this distance lies between $r - 0.5$ and $r + 0.5$, we calculate how far into that interval it is to find the tone of gray color for antialiasing. If it's even further out, we don't color it at all and it stays black for the background.

7.7 The user and developer experience

7.7.1 Compactness and method overloading

The final aspect to think about, now that the engine itself is fully running, is how the user will interact with it. This is not the Unity game engine, so we won't be building an editor UI. But even with scripting, we want to find a good balance between ease of use and amount of control. To illustrate what we ended up with, let's write a script that simulates a world with two circular bodies connected by a distance constraint, influenced by gravity.

```
public static void main(String[] args) {
        // Initiate world and physics loop
        World world = new World();
        PhysicsLoop loop = new PhysicsLoop(world, 200);

        // Define the bodies
```

```
        Body myBody1 = new Circle(10,new Vector2(-50,0),1);
        Body myBody2 = new Circle(10,new Vector2(50,0),1);

        // Add everything to the world
        world.addBody(myBody1);
        world.addBody(myBody2);
        world.addUniversalForce(new Gravity(50));
        world.addStaticConstraint(new DistanceConstraint(myBody1,myBody2));

        // Run the simulation
        loop.start();
}
```

This is very little code to run an entire simulation. The syntax is intuitive: even those who don't know Java or this library should be able to tell what it does. To achieve this compactness it can also be useful to exploit method overloading. Java (and many other languages) allow you to define one method in multiple ways, mainly with different types and amounts of arguments. Take the constructor for the *DistanceConstraint* as an example: in the above snippet the most simple one is used, where it automatically assumes that you want to fix the distance between the centers of the bodies at their current distance. There are more complicated constructors available for specifying the exact points on the bodies that should be connected and at which distance. This principle also applies to the constructors for the body types. This example did not specify any velocity or rotational information for the circles, because those can be set with increasingly expansive constructors that are also available.

7.7.2 Useful (extra) features

With this type of library, a physics engine, it is fair to expect many users will also be developers looking to expand upon your idea or to create games and animations with it. That's why including bonus features you absolutely don't need for the main simulation, isn't a bad idea. For example, we added the ability to turn off a body's collisions and the ability to freeze a body - meaning its velocities are always reset to zero. While freezing objects or turning their collisions off is far from physically meaningful, it can be meaningful to certain projects like games needing stationary floors and walls.

It should be mentioned that when adding frozen bodies, it is not enough to simply reset their velocities to zero each frame and ignore position updates. When other objects collide with a frozen wall and this is not communicated, the flying object assumes a physical collision will occur and some of its momentum and energy is transferred to the wall. Since the wall is frozen, this obviously doesn't happen and thus our flying object can seem to strangely lose all of its momentum when hitting the boundary. To fix this, we need to treat frozen objects as having an infinite mass - this is how they would be frozen physically - which is important to constraints in collisions as the accompanying equations often use inverse mass ($\frac{1}{m}$) and inverse inertia ($\frac{1}{I}$) to divide energy or momentum between objects. Making these inverses return zero for frozen objects fixes the problem, and the system conserves momentum and energy for the non-stationary object.

7.7.3 Tick listeners

A final feature giving the user access to the simulation is the introduction of tick listeners. If the user wants to create interactive simulations, they'll want to run some of their own code each time the world takes a step through time. To achieve this, we can let them implement their own tick listeners as inheritors of the *TickListener* interface. Then they can add this to the physics

world, which will run their piece of code at the start or end of each frame. The associated *PhysicsLoop* object also keeps track of keyboard and mouse presses by default, meaning the user can develop full games to their liking. A simple (included) example is the *CameraController* tick listener, which lets you use the arrow keys and scroll wheel to control the camera's position and zoom level during a simulation.

Chapter 8

Advanced engine features

Now that the baseline physics is fully up-and-running, we can dive deeper into advanced features. For most applications, what we have so far is enough. But if you want to simulate shapes with welds, revolute joints, have more realistic forces or interactions, more math, physics, and code is required!

8.1 Welded bodies

So far, we've completely ignored the ability to weld existing shapes together so they can form more complex ones. This includes the concave shapes we've been fearing, but also any shape like a Tetris L-piece or even a stick figure. This is because simple (convex) bodies pose less of a mathematical challenge. They respect the Separating Axis Theorem and are easy to render using scanline rasterization. A real physics engine however wouldn't deserve its title if it couldn't simulate something simple like an L-piece properly. By supporting welding for concave shapes we can create almost any 2D shape, with the exception of non-circular curved shapes like ovals - those would require a full generalization.

8.1.1 Welds using distance and angle constraints

Considering the L-piece example mentioned just now, one might immediately suggest applying welds as literal constraints on two bodies: take three squares and stitch them together using two distance constraints at each edge! While this idea works perfectly in theory - enforcing points to be at a zero distance counts as welding - it completely fails in practice. The first problem is that the combined bodies still collide with each other, which causes trouble when they're always touching. But even if you implement collision exclusion rules to stop those specific bodies from performing collisions, the instability remains. The root cause is that all of these two distance constraints - or equivalently one distance constraint and one angle constraint - are fighting each other every single frame. They oppose each other and respond by imposing heavy lever effects, with torques that build up until our amalgamate objects rocket off into oblivion. The fact that we're basically imposing zero-distance constraints also does not help, as this means any slight deviation is immediately punished - and zero distance is a strict request.

8.1.2 Bodies and fixtures

So we can't implement welds, but we *can* fake them. Instead of simulating two bodies separately and forcing them to be stuck to each other, we'll program them to be stuck to each other and simulate their behavior on a combined level. If a part of the weld experiences a force, we can pass it on to the total welded body. Detecting collisions is still performed per component, but an actual collision is solved on the combined body using the garnered information.

To be more specific, we will still use the Body class as the main carrier of information like mass, position, velocity, etc. but it now consists of one or more objects called Fixtures. These are the (convex) shapes composing the full body. Our Body class knows where they are positioned relatively to each other, such that it can construct their full combination as a grand shape during rendering or broad-phase collision detection. The Body can use each Fixture's mass, inertia and center of mass to deduce its own total mass (just the sum), total inertia (using a sum and the parallel axis theorem) and total center of mass (using averaging). That total center of mass is still the internal position reference for the Body, which will now store vectors pointing from there to the separate Fixtures' centers of mass, for later use in solving.

The system proposed and implemented here - with a divide between bodies and fixtures - is heavily inspired by a similar system in other 2D physics engines such as Box2D.

8.1.3 Transferring forces, collisions, and constraints

To get the full picture of how bodies can be comprised of fixtures, let's go in-depth per segment of the solver.

- **External forces**: these always act on a full body, not some fixture or part of the body. This means they can still be tracked by the Body class and no real architecture changes are required.
- **Static constraints**: these are also purely functional at the body level. For a distance constraint we just specify the bodies and points on those bodies. No part of the solver cares about that point landing on a specific fixture or part of the full body!
- **Collisions and their dynamic constraints**: we can combine each fixture's AABB by taking their union, yielding an exact AABB for the full body. This can be used in the broad phase, to filter down possible collisions. When two bodies might be colliding, we move down to the fixture level and do a simple AABB overlap check for all pairs of fixtures (between the two bodies). From this we may conclude that a fixture of the first body is possibly overlapping with a fixture of the second body. We then run narrow-phase detection for each of these fixture-fixture candidates, exactly like before using SAT - since each fixture is convex. This yields either a denial of collision or a confirmation, along with all necessary information. Most of this information needs no converting: contact point, penetration depth, normal vector, tangent vector, ... But the relative vectors that would usually point from the fixture's center to the contact point, now need to be converted to relative vectors from the parent body's total center of mass to that same contact point. Once that's all said and done, we can apply the corresponding contact and friction constraints on the parent body level. This makes sense: we have all the necessary information about the collision meaning the shape of the body no longer matters to our solver, everything can now be evaluated for the welded body.

- **Rendering**: the parent body simply calls upon all of its fixtures to render themselves. The fixtures can use their own shape in combination with the parent's position and relative fixture vector to determine which rendering method to call.

8.1.4 Engine updates

In the engine, this change means that the Body class is no longer abstract. This is because the earlier abstraction was based purely on the system not knowing a body's shape in advance - which was required for mass, inertia, COM, ... calculations. But from now on, the shape will be determined by associated fixtures, meaning all information can be pulled from them. Technically this means the user can define their own Body instance by hand, specifying Fixture objects and their relative positions, but this is incredibly tedious. Instead, we will rehash the subclasses of Body like Circle and Polygon, to automatically compose and align these fixtures based on the user's request. The Fixture class itself *will* be abstract, much like the old version of the Body class, with the same subclasses for Polygon and Circle shape fixtures. So when you create a Polygon (subclass of Body) instance, the engine should first determine whether your polygon is convex or concave. If convex, it simply returns a Body that consists of one (convex) Polygon fixture, like normal. If concave, it should split your polygon into multiple convex parts as fixtures, aligned relatively to form the polygon you requested. When you create a Circle (subclass of Body) instance, it just spits out a Body comprised of a single fixture: a Circle fixture. This means we don't actually need to change that much! We're just stitching old shapes together to make new ones, after all.

8.1.5 Dividing concave polygons into sets of convex polygons

WIP

8.2 Revolute pin joints

Another important feature to consider is the addition of revolute pin joints. Despite the complicated name, this simply means connecting bodies at one common point by metaphorically shoving a pin through them. Physically this means that the affected points on both bodies will always remain at the exact same position, but the bodies themselves are free to revolve around those points. It works just like the joints in your body, for example allowing your lower and upper leg to move separately while keeping their connection - the knee - at the same position. For our engine, this would allow us to simulate more mechanical systems, like the double pendulum.[1]

8.2.1 Why a distance constraint fails

Your first instinct for adding this feature might be 'don't we already have this in the form of that distance constraint'? Although this won't work, it's a great intuition since the reason why it fails is quite bizarre. The distance constraint acts as one degree of limitation - since everything is handled via scalars - and it only forces two points at some distance - not to be at

[1] The double pendulum can also be simulated with only a distance constraint, if you place one frozen body as a hinge, then a free second body at fixed distance from the hinge, then a free third body at fixed distance from the other free body. This new constraint however will allow us to do this for full rods between the pendulum points instead of only having masses at the joints.

the same position. Of course, forcing two points at the same spot does imply their distance is zero, but the way the distance constraint is implemented makes it calculate some normal vector connecting the points in order to rule out relative velocity in that direction. If you take the limit of this distance constraint functionality to distance equaling zero, that normal vector becomes impossible to define - dominated by numerical instability - causing it to send objects flying.

8.2.2 Two degrees of constraint

A proper joint constraint needs to enforce two degrees of limitation: the positions must be exactly equal, which implies a constraint on both the x and y directions! You could say this is a vector-based constraint instead of a scalar one. Hypothetically one could split this into two scalar constraints in the code, but that isn't so clean. This is because the physics we're about to derive creates coupling between the two directions. The x-constraint in your code would reference y components and vice versa, which just defeats the whole purpose. Instead, we will have to tweak our engine a bit to allow constraints that solve via vectors and not scalars. This is as simple as making the basic *Constraint* class even more abstract - not even specifying what data type J should have.

8.2.3 Mathematical derivation

Choose a common point on two bodies. Seen from both bodies, their coordinates might be $\vec{p}_1$ and $\vec{p}_2$. The constraint simply enforces that

$$\vec{p}_2 - \vec{p}_1 = 0.$$

This positional constraint will be used for Baumgarte stabilization, also using vectors. Taking the derivative yields the velocity-based constraint,

$$\vec{v}_2 + \vec{\omega}_2 \times \vec{r}_2 - \vec{v}_1 - \vec{\omega}_1 \times \vec{r}_1 = 0$$

where $\vec{r}_1$ and $\vec{r}_2$ are the relative vectors pointing from each body's COM (= rotational center) to the connection point. This expression says that the total (vector) relative velocity between these points must be zero ($\vec{v}_{rel} = 0$), as opposed to its normal projection for previous constraint examples.

Since this is a vector-based equation, we can split it into its two constraint rows like so:

$$A_i = \begin{matrix} v_{1x} & v_{1y} & \omega_1 & v_{2x} & v_{2y} & \omega_2 \end{matrix} \\ \begin{pmatrix} -1 & 0 & r_{1y} & 1 & 0 & -r_{2y} \\ 0 & -1 & -r_{1x} & 0 & 1 & r_{2x} \end{pmatrix}$$

Now let's take a look at the original solver equations, derived all the way back in section 4.4.

$$\Delta J_i = -\frac{A_i v^{n+1}}{A_i M^{-1} A_i^T}$$

$$\Delta v^{n+1} = M^{-1} A_i^T \Delta J_i$$

We now note that since we're solving two constraint rows at once, J_i will be a vector of dimensions 2x1. Because of this, the division shown above no longer makes sense. If one were to properly

convert the Gauss-Seidel update for vectors - or just take a look at the order of operations derived there - it would yield

$$\Delta J_i = -\left(A_i M^{-1} A_i^T\right)^{-1} \left(A_i v^{n+1}\right).$$

The update for velocity does not change symbolically, since both A_i and ΔJ_i change dimensions appropriately. Evaluating $A_i M^{-1} A_i^T$ gives us the complex expression

$$A_i M^{-1} A_i^T = \begin{pmatrix} \frac{1}{m_1} + \frac{r_{1y}^2}{I_1} + \frac{1}{m_2} + \frac{r_{2y}^2}{I_2} & -\frac{r_{1x} r_{1y}}{I_1} - \frac{r_{2x} r_{2y}}{I_2} \\ -\frac{r_{1x} r_{1y}}{I_1} - \frac{r_{2x} r_{2y}}{I_2} & \frac{1}{m_1} + \frac{r_{1x}^2}{I_1} + \frac{1}{m_2} + \frac{r_{2x}^2}{I_2} \end{pmatrix}.$$

To reach an expression for ΔJ_i, we now need to find its inverse. For simplicity we will refer to $A_i M^{-1} A_i^T$ as K from now on. It has the general form

$$K = \begin{pmatrix} a & b \\ b & c \end{pmatrix}$$

of which the inverse can be found (via Cramer's rule) to be

$$K^{-1} = \frac{1}{ac - b^2} \begin{pmatrix} c & -b \\ -b & a \end{pmatrix}.$$

This can be used to determine the impulse update as

$$\Delta J_i = -\frac{1}{ac - b^2} \begin{pmatrix} c v_{rel,x} - b v_{rel,y} \\ a v_{rel,y} - b v_{rel,x} \end{pmatrix}$$

and the resulting velocity update as

$$\Delta v^{n+1} = \begin{pmatrix} -\frac{1}{m_1} & 0 \\ 0 & -\frac{1}{m_1} \\ \frac{r_{1y}}{I_1} & -\frac{r_{1x}}{I_1} \\ \frac{1}{m_2} & 0 \\ 0 & \frac{1}{m_2} \\ -\frac{r_{2y}}{I_2} & \frac{r_{2x}}{I_2} \end{pmatrix} \cdot \Delta J_i.$$

For pseudo-velocities in NGS solving, we need to introduce a bias. This is clearly just the positional error, so $C_i = \vec{p}_2 - \vec{p}_1$. It makes sense that this is once again a vector, since the change in the update is

$$\Delta J_i = -\left(A_i M^{-1} A_i^T\right)^{-1} \left(A_i v^{n+1} + \frac{\beta}{\Delta t} C_i\right).$$

8.2.4 Constraint system rework

To deal with non-scalar constraints, we need to rework our engine's structure. Remember that previously, there existed an abstract class *Constraint* which pre-defined how constraints should be represented and solved, specifically for scalar constraints. We then had *StaticConstraint* and *DynamicConstraint* as abstract inheritors of this class.

In the new system, we will still have an abstract *Constraint* class, but it will only pre-define the most basic methods, that being the ones used by the *World* class. Those are as basic as something

like '*solveConstraint()*' which does not define a data type for J or how it should be solved. Then we still do the same split inheritance to *StaticConstraint* and *DynamicConstraint*, but we still don't specify data types. From here we can define everything ourselves. We can directly inherit from *StaticConstraint* to create the revolute pin joint constraint with its vector implementation. We could do the same for each scalar constraint, but that would involve lots of copy-pasted code. Because of this, it's better to inherit one step further into something like *ScalarStaticConstraint* which would include the pre-defined scalar solver from before. By doing this, our old scalar constraint classes just have to modify their parent class, while their code can remain unchanged.

8.3 Realistic air resistance

WIP

Chapter 9

Extra: Extension to 3D

After creating a successful 2D physics engine, we might look to add one dimension and create the same thing in 3D. But don't be fooled by the small increase from 2 to 3, this makes things a lot harder!

9.1 New variables

Moving to three dimensions introduces one new spatial coordinate, which we'll obviously call z. But for rotations we get two new degrees of freedom, resulting in three angles θ, ϕ, and ψ. In classical mechanics these would be the Euler angles: a simple way to describe the current rotation of the system and express rotational velocity in the local frame of reference, via just time derivatives of those angles. Three-dimensional rotations significantly increase the complexity of motion, as the moment of inertia becomes the inertia tensor, where each direction of rotation has its own (mixed) inertia term.

Real 3D rigid-body engines solve rotations in the body's local frame of reference. We still choose the center of mass as our fixed point of rotation, meaning any force can easily be turned into a torque using $\vec{\tau} = \vec{r} \times \vec{F}$. We then use Euler's equation stating $\vec{\tau} = I\vec{\alpha} + \vec{\omega} \times (I\vec{\omega})$. Note that the second term only arises because our frame of reference might not be an inertial one, so it acts as a gyroscopic or Coriolis torque.[7] Once this has been solved for the rotational velocity $\vec{\omega}$, we don't just use three simple angles to represent rotation. We use quaternions.

Getting into how those work would take us much beyond the scope of this course, so for now we'll just say a quaternion is a four-dimensional representation of our rotations, (w, x, y, z). There then exists a simple differential equation to convert $\vec{\omega}$ to the time derivative of the quaternion representation, allowing us to take steps in time:[10]

$$\dot{q}(t) = \frac{1}{2} q(t) \otimes \omega(t)$$

9.2 Forces and constraints

For forces and constraints, the 2D processes remain mostly the same. Adding another dimension just means an extra application of accelerations. For constraints, we need to extend our

expressions $Av = 0$ to the new dimension, but the Gauss-Seidel solver still works just as well. The iterations of velocity and rotational velocity stay the same. Just remember that after the dust has settled, that rotational velocity still needs to be converted to our internal quaternion representation.

9.3 Changes in collisions

A collision between two bodies in three dimensions is much more detailed than in 2D. The actual collision can take on more forms: it could be a single point, a line, or a full plane of collision. Determining which type you're dealing with and selecting an appropriate number of contact points, is key. For a box resting on the floor, choosing 3 or even 4 contact points is required to prevent the box from rotating into the floor like we had similarly in 2D.

9.4 Friction constraints

A final detail to note is how friction becomes a planar force in 3D. Collisions in 3D still have one primary normal vector, but they have a tangential plane instead of a tangent vector. This means we need to incorporate friction that limits motion in this entire plane. To accomplish this, we choose two perpendicular tangent vectors that span the plane (per contact point!) and use them as regular friction constraints. We must also make sure their combined force stays below the friction maximum, so $\sqrt{J_{t1}^2 + J_{t2}^2} \leqslant \mu J_{normal}$.

9.5 Further reading

For those interested in 3D engines, it can be interesting to check out all the different options out there. Examples include Jolt, PhysX, Havok, and Box3D. The latter was announced and released recently by Erin Catto, whose Box2D was a large inspiration for this project and a tool used by countless 2D games.

Bibliography

[1] Joachim Baumgarte. Stabilization of constraints and integrals of motion in dynamical systems. *Computer methods in applied mechanics and engineering*, 1(1):1–16, 1972.

[2] William Bittle. SAT (Separating Axis Theorem), January 2010.

[3] Adhemar Bultheel. *Inleiding tot de numerieke wiskunde*. ACCO, Leuven, Belgium, 2006.

[4] Erin Catto. Physics for game programmers: Understanding constraints, November 2016. GDC 2014 talk, Internet Archive.

[5] Erin Catto. Solver2D, February 2024.

[6] Erin Catto and Crystal Dynamics. *Iterative Dynamics with Temporal Coherence*. June 2005.

[7] Herbert De Gersem and Eef Temmerman. *Klassieke Mechanica*. Acco, 2024.

[8] Carl Friedrich Gauß. Über ein neues allgemeines grundgesetz der mechanik. 1829.

[9] Kenton Hamaluik. Swept AABB collision detection using the Minkowski difference.

[10] Joan Solà. Quaternion kinematics for the error-state KF. Technical report, Institut de Robòtica i Informàtica Industrial, 9 2016.

[11] Unity Technologies. Unity - Manual: Speculative CCD.

[12] Unity Technologies. Unity - Manual: sweep-based CCD.

[13] Newcastle University (unknown author). Physics - Broad phase and Narrow phase. *Game Engineering Masters Degree*.

[14] Wikipedia. Gauss's principle of least constraint — Wikipedia, the free encyclopedia. `http://en.wikipedia.org/w/index.php?title=Gauss's%20principle%20of%20least%20constraint&oldid=1313487261`, 2026. [Online; accessed 07-January-2026].

[15] Wikipedia. Lagrange multiplier — Wikipedia, the free encyclopedia. `http://en.wikipedia.org/w/index.php?title=Lagrange%20multiplier&oldid=1320521305`, 2026. [Online; accessed 07-January-2026].

[16] Wikipedia contributors. Karush–kuhn–tucker conditions — Wikipedia, the free encyclopedia, 2025. [Online; accessed 15-January-2026].

[17] Wikipedia contributors. Sutherland–hodgman algorithm — Wikipedia, the free encyclopedia, 2025. [Online; accessed 22-March-2026].

[18] Wikipedia contributors. Insertion sort — Wikipedia, the free encyclopedia, 2026. [Online; accessed 11-July-2026].

[19] Wikipedia contributors. Verlet integration — Wikipedia, the free encyclopedia, 2026. [Online; accessed 11-July-2026].

Acknowledgments

The author would like to thank:

- Erin Catto, for his contributions to the discussion of computational physics and especially physics engines for games.
- professors at KULAK university, for their thought-provoking courses that led to the research and creation of this project. Specifically the courses on classical mechanics, numerical mathematics, and object-oriented programming.
- assistents at KULAK university, for aiding with mathematical questions completely unrelated to the course they were tutoring - and for still showing extensive interest.
- family and friends, for showing mild interest each time they were shown an unrequested progress update.

Acknowledgment of GenAI usage

Generative AI chatbots including Google's Gemini, Anthropic's Claude, and OpenAI's ChatGPT were used several times during the writing of this text. They were utilized to discover external papers, courses, and other material which has been properly cited in bibliography. No statements made by AI were taken as fact without external verification.

KU Leuven Kulak
Faculty of Science & Technology
Etienne Sabbelaan 53, 8500 Kortrijk
Tel. +32 12 34 56 78
vermeerencasper@gmail.com